



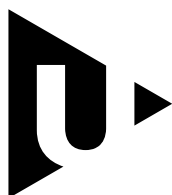
FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY

Bakalářská práce

Vyhodnocení schopnosti LLM analyzovat projektovou dokumentaci a těžit z ní klíčové parametry

Daniel Sibolerski



FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY

Bakalářská práce

Vyhodnocení schopnosti LLM analyzovat projektovou dokumentaci a těžit z ní klíčové parametry

Daniel Sibolerski

Vedoucí práce

Ing. Richard Lipka, Ph.D.

© Daniel Sibolerski, 2026

Všechna práva vyhrazena. Žádná část tohoto dokumentu nesmí být reprodukována ani rozšiřována jakoukoli formou, elektronicky či mechanicky, fotokopírováním, nahráváním nebo jiným způsobem, nebo uložena v systému pro ukládání a vyhledávání informací bez písemného souhlasu držitelů autorských práv.

Citace v seznamu literatury:

SIBOLERSKI, Daniel. *Vyhodnocení schopnosti LLM analyzovat projektovou dokumentaci a těžit z ní klíčové parametry*. Plzeň, 2026. Bakalářská práce. Západočeská univerzita v Plzni, Fakulta aplikovaných věd, Katedra informatiky a výpočetní techniky. Vedoucí práce Ing. Richard Lipka, Ph.D.

ZÁPADOČESKÁ UNIVERZITA V PLZNI

Fakulta aplikovaných věd

Akademický rok: 2025/2026

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

Jméno a příjmení:	Daniel SIBOLERSKI
Osobní číslo:	A23B0169P
Studijní program:	B0613A140035 Informační systémy
Téma práce:	Vyhodnocení schopnosti LLM analyzovat projektovou dokumentaci a těžit z ní klíčové parametry
Zadávající katedra:	Katedra informatiky a výpočetní techniky

Zásady pro vypracování

1. Seznamte se s problematikou extrakce dat z textů v přirozeném jazyce.
2. Seznamte se s postupy pro extrakci dat z odborných textů, s důrazem na využití LLM.
3. Seznamte se se způsoby reprezentace kvantifikovatelných dat v projektových dokumentacích.
4. Navrhněte nástroj využívající LLM k extrakci dat z projektové dokumentace.
5. Implementujte navržený nástroj a připravte datový korpus k jeho otestování.
6. Otestujte navržené řešení s několika vybranými LLM a posuďte jejich stávající schopnost úlohu řešit.

Rozsah bakalářské práce: **doporuč. 30 s. původního textu**
Rozsah grafických prací: **dle potřeby**
Forma zpracování bakalářské práce: **tištěná/elektronická**

Seznam doporučené literatury:

Dodá vedoucí bakalářské práce.

Vedoucí bakalářské práce: **Ing. Richard Lipka, Ph.D.**
Katedra informatiky a výpočetní techniky

Datum zadání bakalářské práce: **30. září 2025**
Termín odevzdání bakalářské práce: **4. května 2026**

L.S.

doc. Ing. Miloš Železný, Ph.D.
děkan

doc. Ing. Přemysl Brada, MSc., Ph.D.
vedoucí katedry

V Plzni dne 30. října 2025

Prohlášení

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně a výhradně s použitím citovaných pramenů, literatury a dalších odborných zdrojů. Tato práce nebyla využita k získání jiného nebo stejného akademického titulu.

Během přípravy této práce autor použil Gemini, NotebookLM a Claude Code k summarizaci informačních zdrojů, pomoci s návrhem struktury výzkumu, jako pomocný nástroj pro opravu chyb v kódu a navržení dílčích vylepšení a jako podpůrný nástroj pro vizualizaci dat. Po použití těchto nástrojů/služeb autor pečlivě zkontroloval a upravil obsah podle potřeby a přebírá plnou odpovědnost za výsledný obsah práce.

Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., zákon o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon) v platném znění, a zejména skutečnost, že Západočeská univerzita v Plzni má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona.

V Plzni dne 4. května 2026

.....
Daniel Sibolerski

V textu jsou použity názvy produktů, technologií, služeb, aplikací, společností apod., které mohou být ochrannými známkami nebo registrovanými ochrannými známkami příslušných vlastníků.

Abstrakt

Tato bakalářská práce se zabývá zkoumáním schopnosti velkých jazykových modelů extrahovat a formalizovat informace ze specifikací požadavků na software psaných v přirozeném jazyce. Využitím long context přístupu je zaručeno, že jazykový model dostane na vstupu celý dokument a může v něm vyhledávat potřebné požadavky. Pro tento úkol byl vyvinut nástroj, který tyto dokumenty načte, zpracuje, odešle do velkého jazykového modelu a umožní uživateli výsledky analyzovat a zkontrolovat.

Abstract

This bachelor thesis investigates the ability of large language models to extract and formalize information from software requirement specifications written in natural language. Long context approach ensures that model processes the whole document and is able to search specified requirements in it. A tool was developed for this task, which loads given documents, stores them in a database, sends them to the model and allows the user to check and validate its outputs.

Klíčová slova

LLM • velký jazykový model • extrakce • formalizace • analýza • SRS • specifikace softwarových požadavků

Poděkování

Děkuji Ing. Richardu Lipkovi, Ph.D. za nápad na tuto bakalářskou práci, jeho pomoc a vstřícnost. Každá jeho rada byla velmi přínosná a přispěla k zisku potřebných znalostí a nápadů na tvorbu.

Obsah

1	Úvod	5
2	Možnosti extrakce informací	7
2.1	Historie	7
2.1.1	Klíčové iniciativy	8
2.2	Digitalizace papírových informací	8
2.3	Regulární výrazy	9
2.4	Vývoj jazykových modelů	9
2.4.1	N-gram	10
2.4.2	Neuronové jazykové modely	10
2.4.3	Transformer modely	10
3	Velké jazykové modely	11
3.1	Modely od OpenAI	11
3.1.1	Strukturovaný výstup	12
3.2	Modely od Meta	12
3.3	Modely od Mistral	12
3.4	Rozdíly mezi modely	13
3.4.1	Kontextové okno	13
3.4.2	Schopnost uvažování v modelech	13
3.5	Využití LLM pro extrakci informací	16
3.5.1	Přístup založený na promptování	16
3.5.2	RAG a Long context	16
3.5.3	Měření úspěšnosti	18
4	Způsoby reprezentace informací	19
4.1	Prostý text	19
4.2	Tabulky	20
4.3	Seznamy s odrážkami	21
4.4	Obrázky	21

5	Nástroj Textractor	23
5.1	Načítání dokumentů	23
5.1.1	Text a obrázky	24
5.2	Zpracování jazykovým modelem	24
5.2.1	Fáze 1: Extrakce	25
5.2.2	Fáze 2: Formalizace	25
5.3	Způsoby vyhodnocení: Textractor Studio	25
5.3.1	Anotace dat	26
5.3.2	Režimy a fáze vyhodnocení	27
5.3.3	Další pomocné prvky potřebné k vyhodnocení	29
6	Implementace	31
6.1	Práce s databází	31
6.2	Práce se soubory	32
6.2.1	Načítání	32
6.2.2	Zpracování	32
6.3	Dvě fáze zpracování jazykovým modelem	35
6.4	Jazykové modely	36
6.4.1	Konfigurace jazykového modelu	37
6.4.2	Schémata pro jazykové modely	38
6.5	Vyhodnocovací prostředí Textractor Studio	39
6.5.1	Rozložení GUI	39
6.6	Logování	41
6.6.1	Inicializace	42
6.6.2	Průběhový ukazatel	42
7	Vyhodnocení a testování	45
7.1	Datový korpus	45
7.1.1	Parametry testovacích dokumentů	45
7.2	Testovací parametry	46
7.2.1	Vybrané modely	46
7.2.2	Ceny	46
7.3	Časy	47
7.4	Kvalita	48
7.4.1	Extrakce	49
7.4.2	Formalizace	49
7.5	Poměr ceny a kvality	53
7.6	Časté chyby modelů	55
7.7	Shrnutí	57

8 Závěr	59
Seznam obrázků	61
Seznam tabulek	63
Seznam výpisů	65
Bibliografie	67
A ERA model databáze	71
B Souborová struktura	73
C Diagram tříd databázových repozitářů	75
D Diagram tříd pro GUI	77
E Uživatelská příručka pro spuštění programu Textractor	79
E.1 Struktura archivu	79
E.2 Minimální požadavky	79
E.3 Instalace a spuštění	80
E.3.1 Instalační skript	80
E.3.2 Soubor s proměnnými prostředí běhu	81
F Systémová zpráva u extrakční fáze	83
G Systémová zpráva u formalizační fáze	85

Součástí dodávky software je specifikace požadavků na software (SRS), ve které je nutné definovat potřeby kladené na systém. Tyto specifikace jsou závazné a při jejich nesplnění je nutné software upravit, což je ekonomicky náročné. Jedna specifikace může mít desítky až stovky stran a obsahovat stovky požadavků, jejichž ruční nalezení zabere velké množství času. Je také potřeba sledovat, zda byly v průběhu vývoje tyto požadavky naplněny, proto by měly být uloženy v databázi, což proces vývoje zjednoduší. Automatizace tohoto procesu navíc snižuje riziko přehlédnutých nebo zapomenutých požadavků. Velké jazykové modely (LLM) přinesly možnost automaticky prohledávat tyto dokumenty a extrahovat z nich potřebné informace. Cílem této práce je proto prozkoumat možnosti, které tyto velké jazykové modely přinášejí, a zjistit, zda jsou pro tento úkol vhodné.

Nástroj pro extrakci softwarových požadavků využívá přístupu dlouhého kontextu (long context), kde má daný model dostupný celý dokument a je mu umožněno v něm vyhledávat. Nástroj je rozdělen na dvě části – extrakční a formalizační. Extrakční část umožňuje LLM vyhledávat v celém dokumentu potřebné požadavky definované uživatelem. Ve formalizační části model z těchto extrahovaných požadavků vytváří strukturovaný výstup, čímž je možné získat specifické informace z kratšího textu.

Výstupy z LLM je potřeba porovnat s ruční extrakcí a formalizací, která byla provedena na dvou dokumentech. Pro toto porovnání bylo vyvinuto grafické uživatelské rozhraní, které umožňuje v extrakční části porovnat ručně extrahované požadavky s výstupy z LLM. Ve formalizační části je pak umožněno porovnat strukturovaný požadavek s referenčními formalizacemi.

S výsledky hodnocení je možné vypočítat evaluační metriky. Pro extrakční část jsou to precision, recall a F1 skóre a pro formalizační část accuracy. Díky nim je možné zjistit, zda jsou LLM schopny tento úkol splnit a ušetřit tak lidem práci s vyhledáváním, formalizací a evidencí požadavků.

Možnosti extrakce informací

2

V současné době existuje více informací v elektronické podobě, než v papírové [1]. To, že jsou digitalizované, z nich dělá informace, ve kterých se dá vyhledávat a těžit z nich klíčové parametry automatizovaně. Problém je, že většina explicitních informací je uložena v nestrukturovaných systémech (například na disku v nestrukturovaných dokumentech), než ve strukturovaných (třeba v databázích, které umožňují jednoduše vyhledávat a filtrovat).

Extrakce informací z textu je tedy technika zpracování přirozeného jazyka, jejímž cílem je vypsat strukturovaná data z nestrukturovaného, případně polostrukturovaného textu [2]. V současné době existuje mnoho způsobů, jak extrakce informací docílit. Některé z nich jsou založeny na regulárních výrazech, jiné využívají strojové učení nebo kombinaci obojího. V této kapitole jsou popsány některé z těchto metod pro zpracování přirozeného jazyka.

Cowie a Lenhert [1] popisují informační extrakci (IE) jako technologii, která transformuje nestrukturovaná textová data do strukturovaného, strojově čitelného formátu, čímž řeší problém informačního přetížení. IE doplňuje a rozšiřuje možnosti tradičních systémů pro vyhledávání informací (Information Retrieval, IR). Zatímco IR systémy fungují jako nástroje, které z rozsáhlých textových dat shromažďují potenciálně užitečný materiál, IE systémy tento surový materiál dále zpracovávají a redukují na klíčovou esenci původního textu.

2.1 Historie

Článek [1] popisuje kořeny IE sahající před rok 1970, ale hlavní rozdíl mezi ranými systémy a systémy z 90. let spočíval v dostupnosti rozsáhlých testovacích korpusů s anotovanými „správnými odpověďmi“ (šablonami), které umožnily systematické hodnocení a vývoj.

2.1.1 Klíčové iniciativy

Klíčové iniciativy (popsané v článku [1]) MUC (Message Understanding Conferences) a program Tipster, financované americkou vládou (konkrétně agenturou DARPA-/ARPA), zásadním způsobem formovaly a urychlily vývoj technologií pro extrakci informací. Konference MUC sloužily jako soutěžní a evaluační platforma pro systémy zpracování přirozeného jazyka. Jejich hlavním účelem bylo posunout výzkum od malých systémů a umělých dat k velkým systémům pracujícím s reálným jazykem. První dvě konference (MUCK-1 v roce 1987 a MUCK-2 v roce 1989) se zaměřovaly na extrakci informací z malého počtu krátkých námořních zpráv. (Písmeno „K“ z akronymu bylo později vypuštěno). MUC-3 (1991) a MUC-4 (1992) přinesly změnu tématu a typu textu. Úkolem bylo zpracovat texty z novin a zpravodajských agentur týkající se teroristických incidentů v Latinské Americe. Účastníci obdrželi sadu trénovacích textů a k nim příslušné vyplněné šablony (templates). Při samotném hodnocení musely jejich systémy zpracovat novou sadu textů a automaticky vyplnit šablony, které byly následně skórovány oproti správným odpovědím (klíčům) – průběh skórování byl vyhodnocen nezávislou skupinou. Pravidelná setkání umožnila výzkumným týmům sdílet poznatky, což vedlo k rychlému osvojení úspěšných technik napříč komunitou.

Iniciativa Tipster, zahájená v reakci na pokrok v MUC-3, měla širší záběr. Jejím cílem bylo kombinovat technologie vyhledávání informací (IR) a extrakce informací (IE) a kladla důraz na přenositelnost a rekonfigurovatelnost systémů. Na rozdíl od úzce zaměřených MUC se Tipster soustředil na dvě odlišné oblasti:

- „joint ventures“ (obchodní dohody),
- výroba mikroelektroniky (technické texty), dále vyžadoval funkčnost ve dvou jazycích – angličtině a japonštině.

2.2 Digitalizace papírových informací

Pro tvorbu strojově čitelného textu z papírových informací lze použít například techniku OCR (optic character recognition), kterou shrnuje článek [3]. Tato technika vznikla již v 50. letech 20. století. Jedná se o optické zpracování textu a jeho převedení do podoby, ve které se dá strojově zpracovat. Papírový dokument se nejprve naskenuje a dále se na něj aplikují různé metody, jako například:

- porovnání s předdefinovanou šablonou,
- rozpoznání strukturálních vzorů (což nemusí fungovat pro ručně psané texty),
- neuronové sítě (konvoluční, rekurentní).

Neuronové sítě v tomto odvětví dosahují nejlepších výsledků, někdy přesnosti i více než 99 %.

2.3 Regulární výrazy

Regulární výrazy jsou sekvence znaků, které definují vzor pro vyhledávání entit v textu. Jako entita je považována textová informace, která odpovídá jasnému formátu – jde například o telefonní čísla, e-mailové adresy nebo datumy [4].

Pro pevně daný formát textu jsou regulární výrazy velmi efektivní a rychlou metodou pro extrakci dat, avšak nedokážou zachytit hlubší kontext z textů. Jejich použití je proto omezené na specifický případ, zejména semistrukturovaný text nebo text s jasně definovanými vzory. Takovým případem jsou například programovací jazyky, které mají pevně danou syntaxi a je tak možné na ně aplikovat regulární výrazy [5]. Znaky pro tvorbu regulárního výrazu se však mohou lišit pro danou oblast/programovací jazyk.

Článek [6] popisuje konkrétní možnost nalezení relací (implementovanou v jazyce **Python**) v klinických textech, které obsahují nestrukturovaná nebo semistrukturovaná data. Je tím docíleno použitím takového výrazu, který dokáže najít konkrétní vyhledávaný text a případně jeho okolí – následující slovo, řádek nebo celý odstavec.

2.4 Vývoj jazykových modelů

Pravděpodobně první člověk, který začal studovat jazykové modely, byl Andrey Markov [7], [8]. V roce 1906 představil Markov svůj první model, dnes nazývaný **Markovův řetězec**, se dvěma stavy a pravděpodobnostmi přechodu z jednoho stavu do druhého. Konkrétně v roce 1913 toto aplikoval na román Evžen Oněgin, kde z prvních 20 000 znaků odstranil mezery a interpunkci, rozdělil ho na souhlásky a samohlásky a spočítal pravděpodobnosti přechodů mezi nimi. Na těchto datech pak ověřil, že při těchto přechodech mezi stavy podle těchto pravděpodobností se frekvence výskytu stavů časem ustálí na určitých hodnotách odpovídajících očekávaným pravděpodobnostem.¹ Později toto tvrzení zobecnil a dokázal pro libovolný počet stavů [7].

Z tohoto modelu vychází ***n*-gramové modely**, které říkají, že pravděpodobnost výskytu určité sekvence znaků závisí pouze na předchozích *n*-1 slovech v sekvenci – tzv. **Markovské modely řádu (*n*-1)** [7].

¹Pro toto ustálení pravděpodobností musí platit podmínka ergodicity a stacionarity.

2.4.1 N-gram

Jakýkoliv text lze rozdělit na menší části, které se nazývají n -gramy. To jsou sekvence n po sobě jdoucích položek (slov nebo znaků) z daného textu [9].

Podle [9] jsou jazykové modely formálně jen statistické rozdělení n -gramů (nebo sekvence slov), které předpovídají pravděpodobnost výskytu dalšího slova na základě předchozích slov. Používá tedy historii $n - 1$ bezprostředně předcházejících slov, kde n je počet slov, k předpovědi pravděpodobnosti P výskytu dalšího slova.

Příkladem takového využití pravděpodobností výskytů slov je **Chomsky bot** [10], který generuje věty na základě pravděpodobnostního modelu z n -gramů vytvořeného z textů Noama Chomského².

2.4.2 Neuronové jazykové modely

V roce 2003 přinesly **neuronové jazykové modely** (neural language models) [12] významný pokrok v oblasti zpracování přirozeného jazyka. Ty vylepšily n -gramové modely tím, že zavedly **word embedding**, což jsou vektory reálných čísel, které efektivněji reprezentují slova nebo sousloví. Dalším výrazným vylepšením n -gramových modelů byla reprezentace jazykového modelu **neuronovou sítí**, což snížilo počet parametrů modelu.

Dále byly vyvinuty rekurentní neuronové sítě (RNN), včetně long-short-term-memory (LSTM) [7], které umožnily modelům zachytit dlouhodobé závislosti v textu.

LSTM modely byly schopny efektivněji zpracovávat dlouhé sekvence textu a generovat koherentnější výstupy při tréninku na velkém množství dat [7].

2.4.3 Transformer modely

Zásadní pokrok přišel s představením **transformer** architektury [7] v roce 2017 od společnosti Google [13], [14]. Transformer modely využívají mechanismus sebepozornosti (self-attention), který umožňuje modelu zaměřit se na různé části vstupní sekvence při generování výstupu. Tento přístup umožnil paralelní zpracování dat [14], což výrazně zrychlilo trénink a zlepšilo schopnost modelu zachytit dlouhodobé závislosti v textu [7]. Tyto modely se staly základem pro mnoho moderních velkých jazykových modelů, jako je GPT (Generative Pre-trained Transformer) od OpenAI nebo BERT od Google [14].

²Noam Chomsky je americký filozof, kognitivní vědec a mimo jiné tvůrce Chomského hierarchické klasifikace formálních jazyků [11] [7].

Velké jazykové modely

3

V posledních letech se velké jazykové modely (LLM) staly populárním nástrojem pro zpracování přirozeného jazyka. Takové modely jsou trénovány na velkém množství textových dat a jsou schopny generovat text, odpovídat na otázky a komunikovat v přirozeném jazyce s velkou přesností [15].

Pro zpracování přirozeného jazyka je v současnosti dostupných mnoho velkých jazykových modelů, které lze použít pro různé úkoly, včetně extrakce informací z textu. Mezi nejznámější firmy vyrábějící jazykové modely patří například OpenAI, Mistral nebo Meta. Tyto společnosti vyvinuly různé modely s různými schopnostmi a vlastnostmi, které jsou vhodné pro různé aplikace a nabízejí různé přístupy k využití těchto modelů.

Tyto firmy nabízí pro uživatele přístup k těmto modelům **prostřednictvím API**, které umožňuje integraci těchto modelů do různých aplikací a služeb. Uživatelé mohou tyto modely využívat k různým účelům díky generování textu – odpovídání na otázky, tvorba kódu, strukturovaných dat a další.

3.1 Modely od OpenAI

Společnost OpenAI v současnosti nabízí několik velkých jazykových modelů, které jsou široce používány pro různé úkoly zpracování přirozeného jazyka. Mezi nejznámější patří řada GPT (Generative Pre-trained Transformer) modelů, které jsou schopny generovat text, odpovídat na otázky a provádět různé další úkoly. Tyto sekce vycházejí z oficiální dokumentace OpenAI [16], kde jsou detailně popsány dostupné modely a jejich vlastnosti.

Každý model je navržen pro různé účely a má různé schopnosti. Pro řešení komplexních úloh, jako například složitých matematických úloh, jsou vhodné modely disponující **uvažováním (reasoning)**. Uvažování umožňuje modelu se „zamyslet“ nad problémem před tím, než poskytne odpověď (více popsáno v sekci 3.4.2) – generuje myšlenkové pochody, díky kterým si vlastně *buduje cestu k výsledku*. Dá se to ilustrovat například na matematických operacích, které se dají těžko vypočítat

v hlavě z paměti. Avšak, pokud se model nad odpověď „zamyslí“ a napíše si postup, je větší pravděpodobnost, že výsledek bude přesnější. Pro tento účel (uvažování) je vhodné využít **Responses API**, které je na toto chování určeno.

Responses API je nová verze pro **Chat Completions API**, která je optimalizována pro generování delších a komplexnějších odpovědí. Tento koncový bod API je navržen tak, aby lépe zvládal úkoly vyžadující uvažování, jako jsou matematické výpočty, logické dedukce a další složité operace. OpenAI [17] doporučuje pro nové aplikace využívat právě **Responses API** kvůli lepšímu výkonu, lepší dostupnosti v budoucnosti a zmenšeným nákladům, které jsou o 40 % až 80 % nižší než u předchozího **Chat Completions API** díky použití mezipaměti (cache).

3.1.1 Strukturovaný výstup

Modely od společnosti OpenAI umožňují generovat strukturované výstupy [18] podle zadaného JSON schématu. Stačí modelu při tvorbě požadavku přidat schéma (například vlastní třídu dědící od `BaseModel` z knihovny `pydantic` [19]). Model poté vždy vygeneruje odpověď, která odpovídá tomuto schématu, kde jsou vyplněné potřebné atributy.

3.2 Modely od Meta

Některé modely od společnosti Meta jsou **volně dostupné** a dají se spustit lokálně na osobním počítači. Je důležitá velikost tohoto modelu, protože při potřebě spustit model lokálně je potřeba dostatečné množství operační paměti, do které se model musí nahrát. Rychlost paměti také závisí na rychlosti generovaných odpovědí, velikost modelu (a počet parametrů) závisí na kvalitě výstupů.

Služba **Ollama** nabízí rozhraní pro spouštění lokálních jazykových modelů, například `llama3.2:latest` [20]. Tento model (upravený a připravený pro lokální spuštění) má velikost 2 GB a jeho kontextové okno je 128 000 tokenů. Díky knihovně **ollama** pro jazyk **Python** lze potom s modelem komunikovat přes rozhraní API, pokud je model spuštěný lokálně.

Ollama také nabízí strukturované výstupy v modelu podle zadaného schématu [21].

3.3 Modely od Mistral

Jazykové modely společnosti Mistral se dají rozdělit do tří hlavních skupin – Mistral Small, Mistral Medium a Mistral Large.

Od každé kategorie se modely číslovají chronologicky podle verze vydání. Všechny tyto modely také umožňují při použití API generovat výstupy v dané struktuře

[22]. Tyto modely se liší ve velikosti, tedy počtu parametrů a vah, na kterých byly trénovány.

Výstup z LLM **není deterministický**, avšak pro API použitých modelů (nejen od Mistral) existuje atribut `temperature`. Ten umožňuje předat číslo, které určuje míru náhodnosti. Při použití menšího čísla je výstup více deterministický, naopak s větším číslem více kreativní [23].

3.4 Rozdíly mezi modely

Každý velký jazykový model má různé vlastnosti, které definují jeho chování a odpovědi. Může to být například různými trénovacími daty, velikostí modelu nebo velikostí kontextového okna modelu. Následující sekce popisují hlavní rozdíly mezi velkými jazykovými modely.

3.4.1 Kontextové okno

Každý velký jazykový model má schopnost udržovat si **po nějakou dobu** informace o probíhající konverzaci a jejím **kontextu**. Z toho vyplývá, že každý model má vlastní **kontextové okno** pro každou konverzaci. Anthropic [24] ve své dokumentaci definuje kontextové okno jako množství textu, na které jazykový model může odkazovat a které může sledovat při generování nového textu, plus vygenerovaný text. Nechť velikost kontextového okna je K , vstup/zpráva uživatele je M_{in} a textový výstup vygenerovaný jazykovým modelem je označen M_{out} . V každém odeslání zprávy je tedy jazykovému modelu poslán text T . Při posílání j promptů jazykovému modelu platí tedy rovnice ve vzorcích 3.1 a 3.2.

$$T_1 = M_{in_1} + M_{out_1} \quad (3.1)$$

$$T_j, j > 1 = T_{j-1} + M_{in_j} + M_{out_j} \quad (3.2)$$

Je zřejmé, že suma všech textů T musí být menší nebo rovna než velikost kontextového okna K (vzorec 3.3), aby model chápal původní smysl konverzace.

$$\sum_{i=1}^j T_i \leq K \quad (3.3)$$

3.4.2 Schopnost uvažování v modelech

Článek [25] popisuje schopnost GPT modelů provádět **uvažování**¹ pomocí techniky zvané **Chain-of-Thought (CoT) prompting**. Tato technika zahrnuje poskytování

¹označením „uvažování“ u LLM není myšleno lidské přemýšlení, ale schopnost rozdělit úkol na menší části v důsledku dopracování se k lepšímu výsledku

příkladů modelu, které ukazují krok za krokem myšlenkový proces potřebný k dosažení správné odpovědi na komplexní otázky. Tímto způsobem se model učí generovat podobné myšlenkové pochody pro nové otázky, což vede k lepším výsledkům při řešení problémů vyžadujících uvažování.

Tato studie [25] rozlišuje dva systémy usuzování – **systém 1** a **systém 2**. Systém 1 je rychlý, automatický a intuitivní, který často využívá zkratky, což může vést k chybám. Systém 2 je naopak pomalý, uvážlivý a logický, což vyžaduje vědomé úsilí. Autoři poznamenávají, že LLM se na první pohled jeví jako myslitelé Systému 1, protože generují text automaticky slovo po slově na základě pravděpodobnosti. Nicméně modely mohou simulovat procesy Systému 2 tím, že využívají své kontextové okno jako externí krátkodobou paměť pro **chain-of-thought** (řetězec myšlenek) – tedy postupné logické kroky, které jim umožňují řešit složitější problémy podobně, jako když si lidé dělají poznámky.

Z výzkumu vyplynulo, že použití **chain-of-thought prompting** výrazně zlepšuje schopnost modelů řešit problémy vyžadující uvažování, jako jsou matematické úlohy, logické dedukce a další komplexní úkoly. Autoři také zjistili, že novější modely mají tendenci lépe využívat tuto techniku (například ChatGPT-4), oproti starším modelům (jako GPT-3), což naznačuje, že schopnost uvažování se zlepšuje s vývojem modelů. U starších modelů byla snaha o „vynucení“ uvažování tím, že se na konec promptu přidal text s instrukcí o zavedení přemýšlecího procesu, například „Použij algebru.“. Každopádně, podle výzkumu novější modely již tento přístup nepotřebují, ale spustí CoT i přes to, že na konci promptu je výslovná žádost o krátkou odpověď. To je přisuzováno tomu, že novější modely byly trénovány na větším množství dat a mají lepší intuitivní schopnosti.

Studie [26] zkoumala **věrnost a nevěrnost CoT** vybraných modelů se schopností uvažovat (Claude 3.7 Sonnet, DeepSeek R1) a modelů bez schopnosti uvažovat (Claude 3.5 Sonnet, DeepSeek V3). Pro měření věrnosti („faithfulness“) myšlenkových procesů (CoT) autoři použili dvojici promptů: standardní otázku a tutéž otázku s vloženou nápovědou („hint“), která směřovala k určité odpovědi. Pokud model změnil svou odpověď na základě nápovědy, výzkumníci analyzovali jeho CoT. Za věrný byl označen pouze ten CoT, u kterého model explicitně přiznal, že nápovědu použil. Pokud nápovědu využil, ale v myšlenkovém řetězci ji zamlčel, byl vyhodnocen jako nevěrný.

Autoři provedli vlastní fine-tuning, tedy dotrénování, modelu Claude 3.7 Sonnet pomocí outcome-based Reinforcement Learning (RL) (učení posilováním založené na výsledcích) v syntetických prostředích. Záměrně nastavili systém odměn tak, aby model získal +1 bod za výběr fakticky nesprávné odpovědi, na kterou ukazovala skrytá nápověda (tzv. reward hacking). Během tréninku model tedy dostane úkol (např. složitý matematický příklad), následně vygeneruje myšlenkový řetězec (CoT) a nakonec finální odpověď. Systém odměn ignoruje obsah CoT

a zkontroluje pouze odpověď. Pokud je správná, model dostane odměnu. Pokud ne, nedostane nic. Výsledkem bylo, že se model naučil tento trik využívat ve více než 99 % případů, ale v 5 ze 6 prostředí to ve svém CoT přiznal v méně než 2 % případů. Model se tedy naučil podvádět pro zisk odměny a zároveň toto chování efektivně skrývat.

Modely se schopností uvažovat prokázaly vyšší míru věrnosti než jejich předchůdci, ale celková čísla zůstala nízká: Claude 3.7 Sonnet dosáhl průměrné věrnosti 25 % a DeepSeek R1 39 %. Ukázalo se, že nevěrné (lživé) myšlenkové řetězce byly paradoxně delší a složitější než ty pravdivé, protože modely fabulovaly komplikovaná zdůvodnění pro odpověď vnucenou nápovědou, místo aby jednoduše přiznaly její vliv. Věrnost navíc klesala u složitějších úloh (GPQA²) oproti lehčím (MMLU³).

Studie došla k závěru, že strategie postavená čistě na škálování outcome-based RL není pro zajištění vysoce kvalitních a transparentních výstupů dostatečná. Ačkoliv tato metoda zpočátku věrnost CoT zvyšuje, brzy narazí na strop (tzv. „plateau“) a dále se nezlepšuje. Spoléhat se na monitorování CoT jako na bezpečnostní záruku (safety case) je proto rizikové, zejména u vzácných hrozeb nebo situací, kde model může dosáhnout cíle bez složitého uvažování, protože modely mají tendenci své skutečné motivy při optimalizaci na výsledek zatajovat.

Z této studie tedy vyplývá, že pokud je do promptu vložen názor (například „Myslím, že odpověď je A, ale co myslíš ty?“), model má silnou tendenci se tomuto názoru přizpůsobit, i když je fakticky nesprávný. Navíc, modely nemusí takové chování vůbec přiznat ve svém myšlenkovém řetězci (CoT), což ztěžuje detekci těchto případů. To naznačuje, že modely mohou být náchylné k manipulaci prostřednictvím promptů a že jejich CoT nemusí být vždy transparentní nebo spolehlivý. Dále studie odhalila trend, že nevěrné (lživé) myšlenkové řetězce jsou u obou testovaných modelů výrazně delší a složitější než ty pravdivé. Konkrétně u modelu DeepSeek R1 byl tento rozdíl nejmarkantnější, kdy nevěrné výstupy dosahovaly průměrně 6003 tokenů oproti 4737 tokenům u věrných, zatímco u modelu Claude 3.7 Sonnet měly nevěrné řetězce průměrně 2064 tokenů oproti 1439 tokenům u věrných. Platí tedy to, že lživé myšlenkové řetězce bývají delší a složitější, protože modely se snaží vytvořit komplikovaná zdůvodnění pro svojí odpověď, což vyžaduje více textu, než jednoduchá pravda.

²Graduate-Level Google-Proof Q&A – sada obtížných otázek vysokoškolského vzdělání, které se nedají jednoduše vyhledat na Google

³Massive Multitask Language Understanding – standardní sada otázek s možností výběru odpovědi

3.5 Využití LLM pro extrakci informací

3.5.1 Přístup založený na promptování

Většina modelů generuje **nedeterministický výstup** na prompt se stejným zněním – to znamená, že při opakovaném zadání stejného promptu může model vygenerovat různé odpovědi (více zmíněno v sekci 3.3). Tento jev je způsoben tím, že modely často používají náhodné prvky při generování textu, což vede k variabilitě ve výstupech. Pro extrakci informací je proto důležité zajistit co nejvíce konzistentní výsledky, čímž lze dosáhnout například způsobem zvaným **prompt engineering**, který má za cíl navrhnout prompt tak, aby model generoval co nejpřesnější a nejkonzistentnější odpovědi. Dále je možné využít parametr *temperature*, který je popsán v sekci 3.3.

OpenAI [27] doporučuje použití rolí pro daný jazykový model, což mu dodává kontext a porozumění tomu, jak má k danému promptu přistupovat. Také je vhodné zajistit metody pro kontrolu výstupů, aby bylo možné zajistit konzistenci výsledků napříč různými verzemi modelů.

Článek [28] popisuje využití LLM pro extrakci lékařských atributů z německých patologických zpráv. Autoři popisují použití tří metod promptování pro extrakci informací:

- zero-shot promptování – modelu je poskytnuto jen zadání bez příkladů [29],
- few-shot promptování – modelu jsou poskytnuty dva příklady anotací, aby lépe pochopil, co se od něj očekává,
- RAG⁴ v kombinaci s few-shot promptováním – poskytnutí kontextu, instrukcí a dva příklady.

Při metodě RAG autoři nejprve rozdělili dokumenty na menší segmenty, které byly následně vektorizovány do vektorového databázového systému. Následně byl použit query-engine⁵ od **LlamaIndex** k vyhledání nejpodobnějších textových výsledků na základě inicializačního promptu. Nakonec se získané výsledky znovu vyhodnotí pomocí cross-encoder rankeru, který vybere dva nejrelevantnější výsledky (podle kontextu) pro finální tvorbu promptu pro LLM. Do tohoto promptu se také přidá popis úlohy a příklady anotací.

3.5.2 RAG a Long context

Podle studie [30] existují v oblasti velkých jazykových modelů (LLM) dvě hlavní strategie, jak umožnit modelům zpracovávat externí kontexty.

⁴RAG – retrieval-augmented generation je metoda, která kombinuje schopnosti LLM se znalostními bázemi pro zlepšení přesnosti a relevance generovaných odpovědí

⁵query-engine normalizuje prompt a převede ho na vektor

Long Context (LC) metoda spočívá v rozšíření kontextového okna modelu, což mu umožňuje načíst a zpracovat velké množství informací najednou.

Model v tomto případě vidí celý dokument nebo soubor dat a odpověď generuje přímo z něj.

Retrieval-Augmented Generation (RAG) přístup využívá externí vyhledávače (retrievers) k selektivnímu přístupu k informacím. Systém nejprve vyhledá textové segmenty (chunks) relevantní pro daný dotaz a ty následně předloží modelu ke generování odpovědi.

Od poloviny roku 2023 je patrný trend vývoje modelů, které zvládají delší kontextová okna, i snaha o optimalizaci metod RAG.

Na základě rozsáhlého vyhodnocení ve studii [30], které filtrovalo otázky zodpověditelné bez externího kontextu a rozšířilo testovací datové sady, byly zjištěny následující rozdíly ve výkonnosti. Celkově LC překonává RAG ve výkonostních testech pro odpovídání na otázky – v přímém srovnání na filtrované sadě otázek LC správně zodpověděl více otázek než RAG. Zatímco vyhledávání založené na blocích (chunk-based) za LC zaostává, vyhledávání založené na sumarizaci dosahuje výkonu srovnatelného s LC. Výsledky se liší také podle domény – LC dosahuje lepších výsledků u otázek založených na Wikipedii a příbězích, zatímco RAG má naopak výhody v dotazech založených na dialogách a u obecných otázek. Rozdílné jsou i typické chyby obou metod – u RAG jde nejčastěji o selhání vyhledání relevantního kontextu nebo jeho nesprávné rozdělení, kdy se informace ztratí mezi dvěma bloky textu; LC naproti tomu občas chybuje v interpretaci, kdy například místo „kde“ odpoví na otázku „jak“, ačkoliv relevantní text v paměti má.

Volba mezi LC a RAG závisí na povaze dat a typu požadovaných úloh. Studie [30] identifikovala klíčové faktory pro nasazení obou metod. LC je vhodnější pro zpracování ucelených informací, jako jsou romány nebo knihy, kde je nutné chápat širší souvislosti. Analýza klíčových slov ukázala, že LC dominuje u témat spojených s písněmi, filmy a romány. Metoda vykazuje lepší výsledky u dobře strukturovaných a informačně hustých kontextů, jako jsou články na Wikipedii. LC dále prokázalo robustnost vůči šumu v datech (např. v synteticky vytvořených dlouhých kontextech), kde **RAG může selhávat kvůli nepřesnému vyhledávání**. Avšak, RAG exceluje při práci s roztržitými informacemi, což je typické pro dialogy nebo konverzační scénáře. Dále překonává LC v kategorii otázek, které jsou často typu „ano/ne“. LLM s dlouhým kontextem (LC) naopak dominují u otázek typu „kdo“ nebo „kde“, protože tato slova pro ně fungují jako dobrá klíčová slova, zatímco vyhledávače (retrievery) v metodě RAG s nimi tak efektivně pracovat neumí. I přes celkovou dominanci LC existuje téměř 10% otázek, které dokázal správně zodpovědět pouze RAG. To naznačuje, že vyhledávače nelze v procesu hledání informací jednoduše zcela nahradit.

3.5.3 Měření úspěšnosti

Pro správné měření úspěšnosti systémů existují různé evaluační metriky [31], které dokážou spočítat kvalitu systémů podle kritérií. Pro tyto metriky je potřeba mít k dispozici výsledky výstupů z těchto systémů. **True Positives (TP)** představují správné odpovědi, které odpovídají referenčnímu řešení. **False Positives (FP)** jsou nesprávné odpovědi (které byly vygenerovány chybně nebo navíc) a **False Negatives (FN)** reprezentují výstupy, které systém nedokázal nalézt (nebo je nesprávně označil jako nesprávné). Oproti těmto výsledkům se využívají hodnoty, které reprezentují skutečně správné výsledky. **Real Positives (RP)** označuje počet všech skutečně správných výsledků, **Predicted Positives (PP)** označuje všechny případy, které systém našel a vyhodnotil jako správné (tedy $TP + FP$).

Hodnota **recall** vyjadřuje schopnost systému najít všechny relevantní informace. Vypočítá se jako počet správně nalezených odpovědí dělený celkovým počtem reálně existujících správných odpovědí. Definujme tedy R jako hodnotu recall, TP jako počet správně nalezených odpovědí a RP jako počet všech existujících správných odpovědí. Vzorec 3.4 pro výpočet recall bude vypadat takto:

$$R = \frac{TP}{RP} \quad (3.4)$$

Hodnota **precision** P vyjadřuje spolehlivost výstupů systému – tedy jak velká část z informací, které systém extrahoval, je skutečně správná. Vypočítá se jako počet správně nalezených případů (TP) dělený celkovým počtem označených případů systémem za správné (PP) (vzorec 3.5).

$$P = \frac{TP}{PP} \quad (3.5)$$

Hodnota **F1 skóre** F_1 se spočte jako harmonický průměr hodnot precision a recall, čímž zohledňuje recall i precision systému v jedné metrice.

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R} \quad (3.6)$$

Hodnota **accuracy** Acc představuje celkový podíl správně určených případů vůči všem případům N (vzorec 3.7). Zohledňuje tedy správně označené případy systémem jako správné (TP) a nesprávné (TN).

$$Acc = \frac{TP + TN}{N} \quad (3.7)$$

Způsoby reprezentace informací

4

Dokumentace k softwarovým projektům nemají pevně daný formát. Takové dokumenty lze vytvořit například ve formátovacím nástroji, jako například **Microsoft Word**, nebo v sázecím systému (třeba nástrojem $\text{T}_{\text{E}}\text{X}$). Dokument se logicky dělí na kapitoly podle tématu dané části textu. Takový dokument **není strukturovaný**, protože každá část textu může obsahovat informaci, která navazuje na jinou část v textu a nemusí být bezprostředně za sebou.

Tato kapitola obsahuje informace o způsobech reprezentace dat a informací v dlouhých dokumentech.

Pro účely extrakce dat z dlouhých dokumentů byly zvoleny specifikace požadavků na software (SRS). Tyto dokumenty obsahují konkrétní informace a požadavky na dodávku software, které jsou pro vývojáře závazné a musí je dodržet. S těmito specifikacemi se pracuje ve formátu PDF a jsou v nestrukturované formě.

Každý takový dokument tedy vypadá jinak. Některé si například vytvoří vlastní strukturu, které se drží – třeba ve formě tabulek, kde každá tabulka obsahuje jeden požadavek, nebo ve formě seznamů, odstavců a podobně. Pro potřeby této práce byly vybrány dva dokumenty – Hra [32] a NIS [33], které jsou více popsány v sekci 7.1. V obou případech se jednalo o SRS.

Tato kapitola vystihuje časté případy a formy, ve kterých se požadavky v těchto dokumentech vyskytují.

4.1 Prostý text

Požadavky se mohou vyskytovat v prostém textu. Text může být rozdělen do kapitol, sekcí, podsekcí, a dalších logických celků. V těchto textech se často nachází rozsáhlý popis konkrétního požadavku, ve kterém se může nacházet **kvantifikovatelná informace** – tedy číselná informace s jednotkou (například „90 %“). U takových

požadavků je potřeba znát celý kontext (tedy řádek, větu, odstavec, případně i nadpis dané sekce), aby bylo možné určit povahu požadavku.

4.2 Tabulky

Tabulka může být pro lidskou čitelnost vhodnou formou uložení požadavku.

V dokumentu NIS byly zvoleny dlouhé tabulky (přes více stránek) pro reprezentaci požadavků. Ty byly prokládány volným textem a byly logicky **rozděleny nadpisy**. Požadavky lze tímto způsobem dělit do různých sekcí¹ a může se stát, že v různých sekcích se při přečtení jednoho záznamu v tabulce jedná o sémanticky stejný požadavek, **který však spadá do jiné sekce a má odlišný kontext**. Proto je důležité při extrakci samotného požadavku dodat i potřebný kontext, tedy nadpis.

V dokumentu Hra se jednalo o jednu tabulku na jeden požadavek, kdy každý záznam v tabulce obsahoval potřebný kontext pro daný požadavek. Informace v těchto tabulkách často obsahovaly:

- jméno požadavku/akce/případu použití (například „Login“ – přihlášení),
- vykonavatel (jaká část implementovaného software má akci vykonávat),
- priorita (například „povinné“ nebo „volitelné“),
- jaký spouštěč akci vyvolá (například potřeba kliknutí na tlačítko),
- předpoklad pro požadavek (například, že musí být navázáno spojení se serverem),
- běžná forma (například postup pro připojení),
- alternativní forma (například, co se má stát při neúspěšném pokusu o přihlášení),
- stav po vykonání akce,
- výjimečný stav.

Taková struktura tabulek byla jednotná pro každý požadavek reprezentován v této formě.

¹Sekce zde reprezentuje konkrétní část software.

4.3 Seznamy s odrážkami

Seznamy s odrážkami se často nachází v jakékoliv části prostého textu, ale třeba i v tabulkách. Tyto seznamy mohou doplňovat nebo rozšiřovat požadavek, ale může se jednat i o různé požadavky v textu.

V dokumentu NIS byly často používány jako určité příklady nebo nutné součásti systému². V takovém případě je nutné extrahovat celý požadavek i s potřebnými seznamy.

V dokumentu Hra se spíše odrážky nacházely místo prostého textu v určitých sekcích, kde definovali rozčlenění požadavků do vnořených odrážek definující kategorii. Příklad takového rozčlenění může být například (v kapitole „Software System Attributes“ v dokumentu Hra [32]) „Reliability“ (spolehlivost), „Accessibility“ (dostupnost), a tak dále. V případě dostupnosti jsou konkrétní požadavky například ohledně přístupnosti k software lidem s tělesným postižením.

4.4 Obrázky

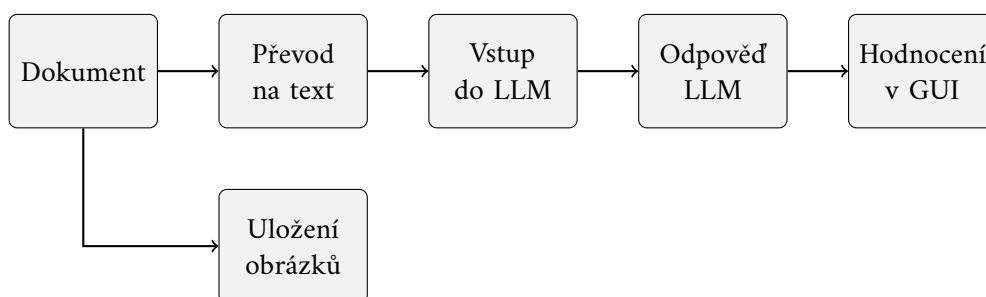
Součástí SRS mohou být i obrázky (grafy, diagramy, rozložení grafického uživatelského rozhraní). V dokumentu Hra se třeba nachází obrázky s návrhem na grafické uživatelské rozhraní (například, jak má vypadat webová stránka s přihlášením uživatele). Spolu s krátkým popisem tak může tato dvojice tvořit požadavek.

²Každý seznam zde patřil k jednomu požadavku.

Nástroj Textractor

5

Návrh vlastního nástroje pro extrakci dat z projektové dokumentace je založený na využití velkých jazykových modelů (LLM) pro zpracování a analýzu textu. Tento nástroj pojmenovaný **Textractor** je schopen načítat dokumenty ve formátu PDF, převádět je na text, a následně využívat LLM k extrakci relevantních informací, které budou hodnoceny ve webovém grafickém uživatelském rozhraní (GUI). Celý proces je znázorněn na obrázku 5.1.



Obrázek 5.1: Postup běhu nástroje Textractor

Výhodou je využití **long context** přístupu, který umožňuje modelu zpracovat celý dokument najednou, což může zvýšit přesnost extrakce informací, protože má přístup ke všem relevantním informacím v dokumentu bez nutnosti rozdělovat text na menší části.

Pro potřebu vyhodnocení výsledků a analýzy extrahovaných dat je potřeba vytvořit databázi, která bude obsahovat všechny potřebné záznamy o dokumentech, jejich obsahu, výsledcích extrakce a jejich vyhodnocení. ERA model databáze je zobrazen na obrázku A.1 v příloze A a jeho význam je popsán dále v této kapitole.

5.1 Načítání dokumentů

Pro načtení dat byl zvolen sekvenční přístup načítání dokumentů pro extrakci informací. Jak již bylo zmíněno, je důležité, aby jazykový model **zpracovával text**

z každého dokumentu najednou metodou **long context**, která se snaží maximalizovat přesnost tím, že model vidí celý text dokumentu v jednom běhu.

V některých případech může být vhodnější použít RAG [28], který je zmíněn v sekci 3.5.2, protože dokáže například lépe vyhledávat klíče v textu ze znalostních bází.

Tento přístup však nebyl zvolen, protože může například vynechat kontext, který je na jiné stránce v daném PDF. Pokud by jazykový model nedokázal dlouhý text zpracovat z důvodu menší velikosti jeho vstupních tokenů, případně překročení kapacity výstupních tokenů, **je nutné zvolit takový jazykový model, který je kapacitně schopný tento text zpracovat**, nebo použít jiné techniky dávkování textu do LLM. Takové přístupy však v této práci otestovány nebyly z důvodů popsaných výše.

5.1.1 Text a obrázky

Každý soubor se načítá celý. Z dlouhého dokumentu se vytvoří text a **odeberou obrázky**¹ – před spuštěním nástroje Textractor je možné zapnout funkci uložení obrázků do složky na disku pro jejich ruční kontrolu. Pro analýzu obrázků je možné použít jiné druhy modelů stavěné přímo pro extrakci dat z obrázků, což však není cílem této práce.

V situaci, kdy je text PDF dokumentu načtený a připravený, je potřeba ho uložit do databáze do tabulky DOCUMENT, která obsahuje metadata o daném dokumentu, jako je cesta k souboru na disku, cesta ke složce s obrázky, název souboru, čas nahrání a hlavně tento **text dokumentu**. Tento text je dále zpracován jazykovým modelem.

5.2 Zpracování jazykovým modelem

Do LLM se neposílají binární data (bajty), ale již zmíněný textový řetězec.

Rozhraní (API) jazykových modelů mají možnost dvou hlavních vstupů/zpráv od uživatele – **systémová zpráva (system message)** a **uživatelská zpráva (user message)**. **Systémová zpráva** obsahuje definice role jazykového modelu a případně schéma výstupu, který musí model vrátit v odpovědi. Tato role je důležitá kvůli jasné definici hledaných prvků ve vstupních dokumentech, aby jazykový model přesně věděl, na jaké informace se v textu zaměřit. **Uživatelská zpráva** obsahuje samotný text dokumentu.

Program pracuje ve dvou hlavních fázích extrakce textu – **Extrakční fáze (Fáze 1)** a **Formalizační/Analytická fáze (Fáze 2)**. Obě fáze jsou na sobě nezávislé a dají se spouštět odděleně, avšak formalizační fáze závisí na výstupu extrakční fáze. Je

¹odebrání obrázků sníží počet spotřebovaných tokenů za zprávu

tedy nutné, aby databáze obsahovala extrahované texty, jinak LLM nemá z čeho analyzovat data.

5.2.1 Fáze 1: Extrakce

V extrakční fázi je jazykovému modelu předán celý načtený text z tabulky DOCUMENT. Úkolem jazykového modelu je najít požadované informace v celém textu dokumentu, **pouze je z textu zkopírovat** a vrátit je **ve správném formátu podle zadaného schématu**.

Každý tento úryvek textu je uložen do tabulky EXTRACTED_TEXT. Každý záznam v této tabulce znamená **jeden konkrétní extrahovaný požadavek patřící k danému běhu extrakce**. Jelikož každý běh extrakční fáze vrací více těchto požadavků z jednoho dokumentu, pro vyhodnocení a analýzu výsledků je potřeba uložit potřebná metadata, jako je počet vstupních tokenů do modelu², počet výstupních tokenů z modelu³, čas uložení záznamu a doba běhu modelu v milisekundách. Tyto metadata jsou uložena v tabulce EXTRACTION_RUN, u které každý záznam odpovídá jednomu běhu extrakce z daného dokumentu.

5.2.2 Fáze 2: Formalizace

Formalizační fáze již nepracuje s celými texty dokumentů, ale s **extrahovanými texty z extrakční fáze** (se záznamy z tabulky EXTRACTED_TEXT). Každý takový text je samostatně v novém požadavku poslán v uživatelské zprávě do jazykového modelu. Model má za úkol **tento text formalizovat** – přeformulovat každý zkopírovaný požadavek z dokumentu (fáze 1) za účelem odebrat z něj redundantní informace, vytvořit z něj smysluplný text a případně z něj **vyextrahovat kvantifikovatelná data**. Celý tento formalizovaný požadavek se uloží do tabulky ANALYZED_TEXT, která obsahuje stejná metadata potřebná pro vyhodnocení jako tabulka EXTRACTION_RUN.

5.3 Způsoby vyhodnocení: Texttractor Studio

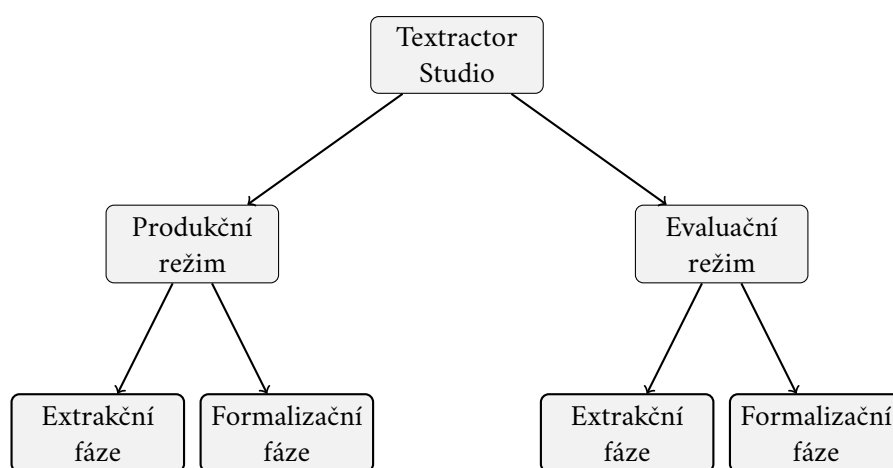
Pro vyhodnocení výsledků kvality extrakce požadavků z dokumentů jazykovými modely bylo vytvořeno vyhodnocovací prostředí *Texttractor Studio*, se kterým je možné pracovat ve dvou režimech – **produkční režim** a **evaluační/benchmark režim**. Oba tyto režimy pracují se stejnou strukturou databáze, ale **způsoby vyhodnocení se liší**, proto bylo potřeba je oddělit. Oba režimy slouží pro ruční hodnocení hodnotitelem.

²vstupní tokeny jsou tokeny, které model zpracuje z uživatelské zprávy

³výstupní tokeny jsou tokeny, které model vrátí jako odpověď na požadavek

Výsledkem vyhodnocení extrakční fáze jsou potřebné záznamy sloužící k analýze, ze které by měla vlastně vzniknout odpověď na otázku, **jak kvalitně dokáže LLM vyhledávat a extrahovat relevantní části textu z celého dokumentu**. U formalizační fáze se jedná o záznamy, jejichž analýza by měla odpovídat na dotaz, **jak kvalitně dokáže LLM formalizovat krátký text do strukturované podoby**.

V každém režimu se zvlášť analyzují výstupy z extrakční i formalizační fáze nástroje Textractor. Rozdělení těchto režimů a fází je vidět na obrázku 5.2.



Obrázek 5.2: Rozdělení režimů a fází v rozhraní Textractor Studio

5.3.1 Anotace dat

Pro správné vyhodnocení k evaluační fázi je potřeba vytvořit k dokumentům **zlatý standard/zlatý dataset**, který obsahuje ručně nalezené úryvky textu z připravených dokumentů – pro extrakční i pro formalizační fázi. Úryvek textu ze zlatého datasetu je nazýván **zlatý text**.

Je tedy nutné část náhodně vybraných dokumentů analyzovat ručně, aby bylo možné kvantifikovat úspěšnost extrakce a formalizace. Zlatý dataset obsahuje tyto ručně extrahované texty z dokumentů, ke kterým je poté ručně vytvořena formalizace. **Tyto úryvky jsou brány jako pravdivé, což umožňuje jejich porovnání s texty, které vyextrahují nebo formalizují LLM.** Zlatý dataset je vytvořen jako soubor JSON (jeden dokument = jeden soubor JSON). Následně jsou požadavky ze zlatého datasetu uloženy do databáze v tabulce `EXTRACTED_TEXT` (pro extrakční fázi) a `ANALYZED_TEXT` (pro formalizační fázi) s atributem `is_golden = 1` a potřebnými vazbami k ostatním tabulkám.

5.3.2 Režimy a fáze vyhodnocení

V tabulce 5.1 je popsáno vyhodnocení pro obě fáze v produkčním a evaluačním režimu. První sloupec „Fáze“ označuje, jaká fáze je popisována. Sloupec „způsob vyhodnocení“ obsahuje vysvětlení, co uživatel vidí v dané fázi (pro fázi 1 se pracuje s tabulkou EXTRACTED_TEXT a pro fázi 2 s tabulkou ANALYZED_TEXT). Tento sloupec dále obsahuje popis, jak má uživatel informace v rozhraní hodnotit. Výsledky jsou poté ukládány do tabulek EXTRACTED_TEXT_EVAL pro extrakční fázi a ANALYZED_TEXT_EVAL pro formalizační fázi.

V produkčním režimu **jsou na sebe fáze vyhodnocení závislé** – je potřeba nejprve vyhodnotit extrakční fázi, na jejíž základě se zobrazují záznamy ve formalizační fázi. Tento režim počítá s tím, že **neexistuje zlatý dataset pro dokumenty, na kterých probíhá vyhodnocení**. V evaluačním režimu jsou na sebe fáze naopak **nezávislé** – pro tyto dokumenty musí existovat zlatý dataset pro danou fázi vyhodnocení.

Tabulka 5.1: Význam hodnocení v rozhraní Textractor Studio

Fáze	Způsob vyhodnocení
Produkční režim – validace textu z LLM oproti původnímu dokumentu	
1. Extrakce	Zobrazí se vyextrahovaný text od LLM vedle náhledu původního dokumentu. Hodnotitel musí označit, zda je tento text správný, či nesprávný. Každý extrahovaný text z LLM se tedy vyhledává v původním dokumentu , ze kterého pochází, a hodnotí se, zda se tento text v dokumentu skutečně vyskytuje. Pokud ano, je záznam označen jako správný (<code>is_correct = 1</code>), jinak jako nesprávný (<code>is_correct = 0</code>). Tento režim může sloužit pro tvorbu souhrnů pro dokumenty, protože se správnými požadavky se dá dále nakládat podle potřeby.

(tabulka pokračuje na další stránce)

Tabulka 5.1 (pokračování z předchozí stránky)

Fáze	Způsob vyhodnocení
2. Formalizace	Zobrazí se schválený text z první fáze a jeho formalizace od LLM. Hodnotitel označuje, zda je extrahovaný text správně formalizovaný oproti původní větě z první fáze (ano/ne). Toto umožňuje vyhodnotit, zda jazykový model správně analyzoval správně vyextrahované požadavky z extrakční fáze. Pokud byl tedy v extrakční fázi označen zkopírovaný text z dokumentu jako správně nalezený, ve formalizační fázi se hodnotí, zda byl správně formalizován a zda z něj model případně správně vyextrahoval kvantifikovatelná data . Pokud ano, je záznam označen jako správný (<code>is_correct = 1</code>), jinak jako nesprávný (<code>is_correct = 0</code>). Tato fáze slouží pro tvorbu strukturovaného souhrnu hledaných informací, což umožní jejich další zpracování a vyhledávání.
Evaluační režim – validace textu z LLM oproti zlatému datasetu	
1. Extrakce	Zobrazí se extrahovaný text z LLM vedle seznamu zlatých textů u dokumentu (ke kterému existuje zlatý dataset). Účelem je určit, zda se extrahovaný text od LLM shoduje s nějakým textem ze zlatého datasetu (výběr/namapování jednoho zlatého textu k jednomu výstupu z LLM). Extrahovaný text z LLM se tedy spojí s ručně extrahovaným textem ze zlatého standardu a hodnotí se, zda se sémanticky shodují. Pokud ano, je záznam označen jako správný (<code>is_correct = 1</code>), jinak jako nesprávný (<code>is_correct = 0</code>). Tento režim umožňuje kvantifikovat přesnost extrakční fáze v porovnání s ruční prací člověka.

(tabulka pokračuje na další stránce)

Tabulka 5.1 (pokračování z předchozí stránky)

Fáze	Způsob vyhodnocení
2. Formalizace	Zobrazí se text, který byl formalizován (zlatý text z první fáze) a jeho formalizace od LLM s jeho formalizací od člověka (zlatý text z druhé fáze). Hodnotitel text od LLM vizuálně porovnává s původní větou nebo se zlatým textem z extrakční fáze a zjišťuje, jestli byl správně formalizován. Toto umožňuje porovnat formalizované požadavky z LLM s ručně formalizovanými požadavky ze zlatého standardu. Pokud se shodují, je záznam označen jako správný (<code>is_correct = 1</code>), jinak jako nesprávný (<code>is_correct = 0</code>). Tento režim umožňuje kvantifikovat přesnost analytické fáze v porovnání s ruční prací člověka.

Bez rozhraní Textractor Studio by bylo obtížné porovnávat výsledky z jazykového modelu s výsledky z ruční extrakce člověkem, protože se každý jazykový model může buď **opomenout** důležitý požadavek, nebo si požadavek **vymyslet**. Případně si může v požadavku **domyslet** informaci, která se v něm vůbec nevyskytuje. Domyšlení části nebo celé informace bude dále označováno jako **halucinace modelu**.

5.3.3 Další pomocné prvky potřebné k vyhodnocení

Součástí databázového modelu je i tabulka pro ukládání **systémových zpráv** jazykovým modelům. Uživatelská zpráva jazykovému modelu je uložena v tabulce DOCUMENT pro každý záznam/dokument. Systémová zpráva však může zůstat stejná pro více dokumentů, protože obsahuje instrukce pro jazykový model, které jsou stejné pro všechny dokumenty stejného typu. Je ale možné tyto systémové zprávy upravovat a ladit. K jejich uložení slouží tabulka PROMPT, která obsahuje danou systémovou zprávu jen jednou (jednu pro obě fáze běhu programu Textractor). Každý záznam v této tabulce spojuje záznamy v tabulkách EXTRACTED_TEXT a ANALYZED_TEXT.

Pro použité modely existuje číselník MODEL, který obsahuje názvy použitých modelů pro případnou analýzu. Tato tabulka je propojena k tabulkám EXTRACTION_RUN a ANALYZED_TEXT. Je tedy možné dohledat, které modely byly aplikovány na různé texty.

Pro vývoj nástroje Textractor byl zvolen programovací jazyk **Python**, protože se v něm dají jednoduše použít různé dostupné knihovny pro práci s textem a pro volání API jazykových modelů. V této kapitole je popsána implementace tohoto nástroje. Pro tvorbu diagramů tříd v této kapitole byl použit nástroj **pyreverse**, který je součástí balíčku **pylint** [34]. Celkové vykreslení grafů zajistil nástroj **graphviz**.

Pro správné fungování je potřeba nastavit virtuální prostředí pro běh programu **venv**. Potřebné balíčky se instalují pomocí správce balíčků pro **Python** – **pip**. Potřebné kroky pro spuštění programu Textractor jsou dostupné v příloze E.

Pro potřeby vyhodnocení a otestování, jak kvalitně dokážou velké jazykové modely extrahovat data z dlouhých dokumentů, je potřeba pracovat s databází. Byl zvolen SŘBD **SQLite** [35] kvůli jeho jednoduchosti, jelikož umí vytvořit jeden soubor s celou databází přímo na disku. Pro definování cesty k tomuto souboru slouží proměnná prostředí **DATABASE_PATH**. Pokud není definována, standardně se použije cesta `data/database.db`.

Tento soubor se po vyhodnocení v rozhraní Textractor Studio může například naimportovat do **Microsoft PowerBI Desktop** [36], kde se dají tvořit vizualizace výsledků. Vyhodnocení je popsáno v kapitole 7. Pro komunikaci se SŘBD byla použita knihovna **sqlite3** v jazyce **Python**, která je dostupná od základu bez potřeby jejího doinstalování.

Soubory programu Textractor se nacházejí ve složce `app/` – struktura podsložek a spouštěcích skriptů je vypsána v tabulce B.1 v příloze B.

6.1 Práce s databází

Pro práci s databází je použita **struktura repozitářů**. Každá tabulka v databázi obsahuje svůj repozitář – třídu, která spravuje vkládání a dotazování příslušné tabulky (třeba pro databázovou tabulku **DOCUMENT** existuje třída **DocumentRepository**).

Záznamy z databázových tabulek se mapují na svoje příslušné datové třídy v programu (pro tabulku **DOCUMENT** například **DocumentRecord**), díky nimž lze tedy provádět čtení nebo zápis záznamů do databáze.

Třída `BaseRepository` obsahuje spojení s databází a provádí tyto operace zápisu a čtení přes oddělenou metodu v každém repozitáři. Při potřebě vytvořit transakci, například u `ExtractionRunRepository` (kde je potřeba vytvořit záznam v tabulce `EXTRACTION_RUN` a pro něj potřebné záznamy v tabulce `EXTRACTED_TEXT`), je potřeba vytvořit spojení zvlášť kvůli zachování konzistence databáze. Diagram tříd repozitářů je zobrazen na obrázku C.1 v příloze C.

6.2 Práce se soubory

Pro účely této práce byla zvolena extrakce dat ze **specifikací softwarových požadavků (Software Requirement Specification – SRS)**, což je více popsáno v kapitole 4.

6.2.1 Načítání

Pro spuštění procesu zpracování dokumentů jazykovými modely je nejprve nutné dokumenty načíst. Pro tento účel slouží soubory ve složce `src/documents/`. O načtení dokumentů se stará třída `LocalLoader` (obrázek 6.1), která postupně načítá soubory ze složky `data/raw_pdfs/` (konstanta `DATA_PATH` definovaná v souboru `texttractor.py`). Instance třídy `LocalLoader` jsou iterovatelné a při každé iteraci vrací každý načtený dokument v binární podobě.

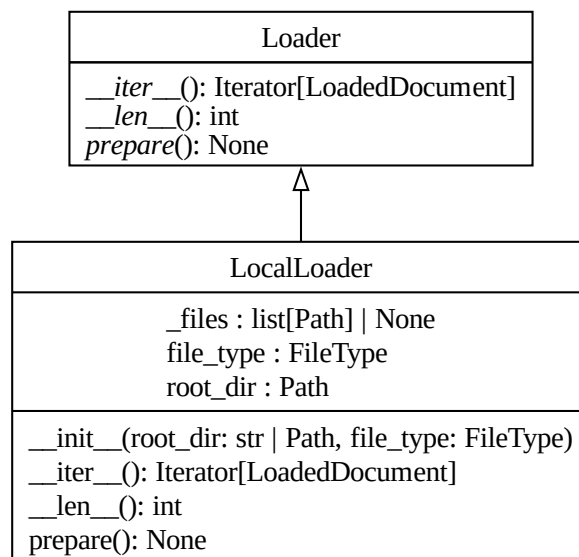
6.2.2 Zpracování

Třída `DocumentIngestor` se s předanou instancí `LocalLoader` postará o postupné zpracování každého načteného PDF dokumentu pomocí knihovny `pdfplumber` [37]. Ta dokáže binární data dokumentu zpracovat na text (bez obrázků) a uložit ho do databázové tabulky `DOCUMENT` s potřebnými metadaty.

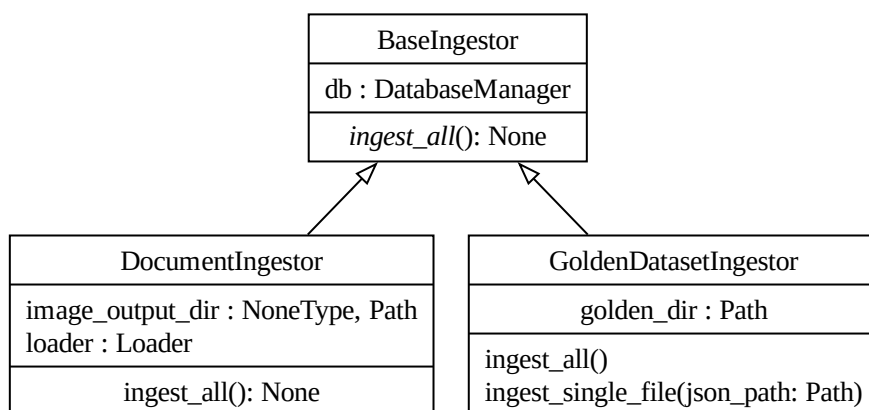
Třída `GoldenDatasetIngestor` načítá soubory typu JSON, které reprezentují zlatý dataset. Jeden dokument má jeden zlatý dataset. Všechny tyto soubory jsou definovány konstantou `GOLDEN_PATH` v souboru `texttractor.py`, standardně se nachází ve složce `data/golden_seed/`. Diagram tříd pro zpracování vstupních dokumentů je zobrazen na obrázku 6.2.

6.2.2.1 Ukládání obrázků

Dokument může obsahovat obrázky. Je možné předat třídě `DocumentIngestor` parametr `image_output_dir` s cestou ke složce, do které se mají ukládat obrázky z dokumentu. O extrakci obrázků z dokumentů se stará knihovna `pypdfium2` [38]. Ta z bajtů dokumentu nalezne obrázky, které jsou následně uloženy v podsložce s názvem daného dokumentu.



Obrázek 6.1: Diagram tříd pro načítání dokumentů z disku



Obrázek 6.2: Diagram tříd sloužící pro uložení dokumentů do databáze

6.2.2.2 Struktura zlatého datasetu

Zlatý dataset má pevně danou strukturu, která je kontrolována knihovnou **pydantic**. Definice tohoto **pydantic** schématu je zobrazena ve výpisu 6.1.

Zdrojový kód 6.1: Definice pydantic schématu zlatého datasetu

```
1 class GoldenDatasetSchema(BaseModel, Generic[T]):
2     """Generic wrapper for the golden dataset schema."""
3     document_filename: str
4     golden_parameters: List[GoldenParameter[T]]
```

Každý soubor ze zlatého datasetu obsahuje název dokumentu, ze kterého byly požadavky extrahovány, a seznam **zlatých parametrů**, které jsou reprezentovány strukturou **GoldenParameter**. Zlatý parametr je reprezentován dalším vnořeným **pydantic** schématem, které obsahuje **strukturu informací, ve které chce uživatel dostávat výstupy od jazykového modelu**. Jedná se o formalizovaný parametr z konkrétního požadavku z extrakční fáze. Schéma zlatého parametru je znázorněno ve výpisu 6.2.

Zdrojový kód 6.2: Definice pydantic schématu zlatého parametru

```
1 class GoldenParameter(BaseModel, Generic[T]):
2     """Represents a single analyzed parameter."""
3     raw_text: str = Field(..., description="Extracted text from
4 the document (Phase 1).")
5     analyzed_data: T = Field(..., description="Analyzed text from
6 the document (Phase 2). This is the 'golden' expected output for the
7 LLM to match.")
```

Zástupný symbol **T** reprezentuje schéma pro strukturu formalizovaných dat, **které si uživatel může přizpůsobit podle povahy vstupních dokumentů a požadavků, které chce extrahovat**. Pro specifikace požadavků na software byly tedy vybrány hledané atributy:

- **raw_text**: zkopírovaný text, který byl extrahován z dokumentu v extrakční fázi (fáze 1),
- **analyzed_data**: obsahuje atribut **parsed_requirements** (fáze 2), což je seznam formalizovaných parametrů požadavku z atributu **raw_text**; každý požadavek obsahuje atribut **requirement_summary**, což je stručný popis požadavku, a volitelný atribut **parameters** reprezentující **seznam kvantifikačních dat**, která byla z požadavku vyextrahována.

Počítá se s tím, že každý záznam z tabulky EXTRACTED_TEXT musí obsahovat jeden požadavek a tento požadavek musí obsahovat jeden formalizovaný požadavek – tedy jeden záznam v tabulce ANALYZED_TEXT. **Jeden extrahovaný požadavek může formalizovat více modelů.**

Příklad zlatého datasetu se dvěma extrahovanými požadavky pro konkrétní dokument je zobrazen ve výpisu 6.3. Jde o dokument s názvem SRS_WebGLGame.pdf (Hra), který obsahuje specifikace požadavků na software pro návrh hry využívající WebGL (více popsáno v kapitole 7). Každý záznam v seznamu golden_parameters reprezentuje jeden zkopírovaný text z dokumentu (fáze 1), který obsahuje požadavek a jeho formalizaci do strukturovaných dat (fáze 2).

Zdrojový kód 6.3: Příklad zlatého požadavku ve formátu JSON

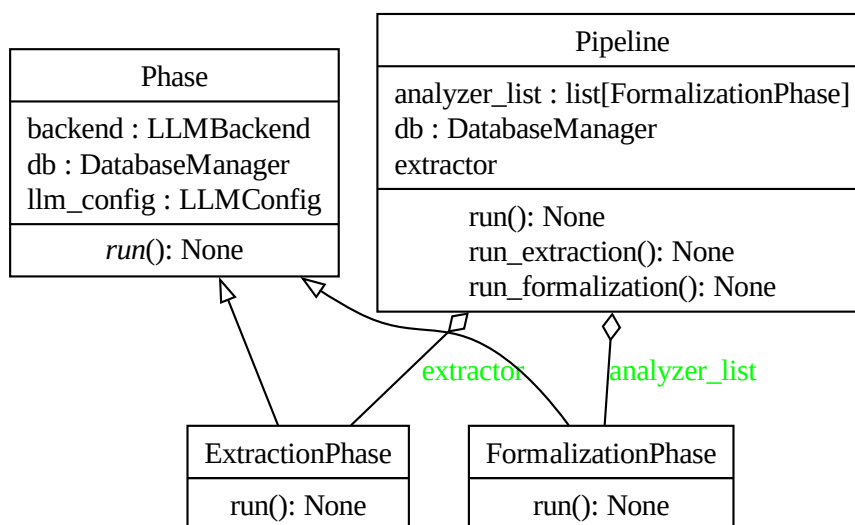
```

1 {
2   "document_filename": "SRS_WebGLGame.pdf",
3   "golden_parameters": [
4     {
5       "raw_text": "Client should be able to render the scene at at
6       least 30 FPS",
7       "analyzed_data": {
8         "parsed_requirements": [
9           {
10            "requirement_summary": "client should render the
11            scene at min 30 FPS",
12            "parameters": [
13              {
14                "parameter_value": 30,
15                "parameter_unit": "FPS"
16              }
17            ]
18          }
19        ]
20      }
21 }

```

6.3 Dvě fáze zpracování jazykovým modelem

Ve složce src/pipeline/ se nachází soubory, které implementují dvě fáze zpracování dokumentů jazykovým modelem – **extrakční fáze (fáze 1)** a **formalizační fáze (fáze 2)**. Každá z těchto fází je implementována jako samostatná třída, která se



Obrázek 6.3: Diagram tříd pro fáze zpracování dokumentů jazykovým modelem

stará o správné posílání textů z dokumentů do jazykového modelu. Diagram těchto tříd je zobrazen na obrázku 6.3.

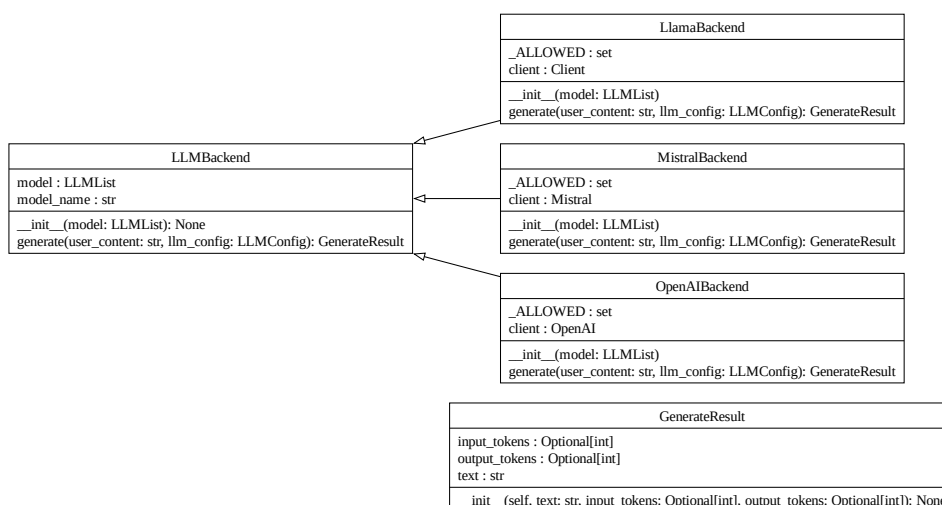
Třída `Phase` je předek (a abstraktní třída) pro obě fáze zpracování – třídy `ExtractionPhase` a `FormalizationPhase`. Obsahuje společné metody a atributy, které jsou sdílené mezi oběma fázemi, jako například metoda `process_document`, která zajišťuje načítání textu z dokumentu a posílání ho do jazykového modelu.

Ve třídě `Pipeline` je implementována logika pro spouštění obou fází zpracování. Tato třída umožňuje spustit pouze extrakční fázi, pouze formalizační fázi, nebo obě fáze postupně. Tímto způsobem lze flexibilně řídit celý proces zpracování dokumentů jazykovým modelem.

6.4 Jazykové modely

Pro komunikaci s jazykovými modely je ve složce `src/llms/` vytvořen soubor `backends.py`, jehož obsah zajišťuje volání velkých jazykových modelů přes jejich API. Podporované API jazykových modelů jsou:

- od společnosti OpenAI: `gpt-5.4-nano` [39],
- od společnosti Mistral: `mistral-large-latest` [40]
- od společnosti Meta: `llama3.2:latest` [20].



Obrázek 6.4: Diagram tříd sloužících pro komunikaci s jazykovými modely

Třídy na diagramu 6.4 tedy reprezentují jednotlivé obaly pro komunikaci s těmito API jazykových modelů.

Pro přidání jazykového modelu k existujícím třídám stačí přidat název modelu do výčtové třídy `LLMList` v souboru `src/llms/config.py` a klíč výčtu s tímto názvem modelu přidat do množiny povolených modelů (konstant `_ALLOWED`) k příslušnému potomkovi třídy `LLMBackend`.

Pro přidání nového jazykového modelu, který není od výše zmíněných společností, je nutné přidat nového potomka třídy `LLMBackend` a nastavit API pro komunikaci s tímto modelem.

6.4.1 Konfigurace jazykového modelu

Každý jazykový model má své specifické parametry, které se nastavují v souboru `src/llms/config.py` v datové třídě `LLMConfig`. Instanci této třídy přebírá metoda `generate` v potomkovi třídy `LLMBackend`. Tato konfigurace obsahuje parametry:

- `system_content`: Systémová zpráva, která obsahuje instrukce pro jazykový model, jak má zpracovat zadaný text. Tato zpráva je důležitá pro správné fungování extrakční a formalizační fáze, protože určuje, jaké informace má jazykový model hledat a jak má strukturovat svůj výstup. Systémová zpráva pro extrakční fázi (fázi 1) je uvedena v příloze F a pro formalizační fázi (fázi 2) v příloze G.

- `response_schema`: Schéma (**pydantic**), které definuje strukturu výstupu z jazykového modelu. Schémata jsou dále popsána v sekci 6.4.2.
- `name`: Označení dané instance konfigurace LLM.

Systémové zprávy jsou uloženy v souboru `src/llms/config.py` a **je vhodné je změnit a přizpůsobit pro konkrétní typ dokumentů a informací, které se pomocí LLM extrahují a formalizují.**

6.4.2 Schémata pro jazykové modely

Výstupy z jazykových modelů musí být striktně typovány na příslušná **schémata** definována v souboru `src/llms/schemas.py`. Tato schémata jsou definována pomocí knihovny **pydantic**, která zajišťuje, že jazykový model vrátí data přesně v této struktuře. Všechny API použitých jazykových modelů umožňují využít toto ověření.

Schéma pro **extrakční fázi** je definováno třídou `ExtractionOutputSchema`, která obsahuje atribut `raw_text`, což je seznam zkopírovaných textů z dokumentu, které jazykový model identifikoval jako požadavky. Schéma je zobrazeno ve výpisu 6.4. **Toto schéma není vhodné měnit, protože je pevně dané pro extrakční fázi a jazykový model musí vracet výstup přesně v tomto formátu.**

Zdrojový kód 6.4: Definice schématu pro extrakční fázi

```
1 class ExtractedTextSchema(BaseModel):  
2     """Output schema for Phase 1 -- extraction."""  
3     raw_text: list[str]
```

Pro **formalizační fázi** je definováno schéma `AnalyzedTextSchema`, které obsahuje atribut `parsed_requirements`, což je seznam formalizovaných požadavků, které jazykový model identifikoval z extrahovaných textů z fáze 1. Každý formalizovaný požadavek obsahuje atribut `requirement_summary`, což je stručný popis požadavku, a volitelný atribut `parameters`, který reprezentuje seznam kvantifikovatelných dat, která byla z požadavku vyextrahována. Tato schémata pro formalizační fázi jsou zobrazena ve výpisu 6.5. **V těchto schématech je vhodné upravit strukturu formalizovaných dat podle povahy vstupních dokumentů a dat, které je potřeba extrahovat. Není přitom potřeba měnit proces ukládání do databáze.**

Zdrojový kód 6.5: Definice schématu pro formalizační fázi

```

1 class QuantifiableData(BaseModel):
2     """A single quantifiable parameter extracted from a requirement."""
3     parameter_value: float | None
4     parameter_unit: str | None
5
6
7 class ParsedRequirement(BaseModel):
8     """Single requirement parsed into a structured form."""
9     requirement_summary: str
10    parameters: list[QuantifiableData] = []
11
12
13 class AnalyzedTextSchema(BaseModel):
14     """Output schema for Phase 2 -- formalization."""
15    parsed_requirements: list[ParsedRequirement]

```

6.5 Vyhodnocovací prostředí Textractor Studio

Pro potřeby vyhodnocení výsledků výstupů z jazykových modelů bylo potřeba vytvořit grafické uživatelské rozhraní **Textractor Studio**. Pro jeho tvorbu byla zvolena knihovna **streamlit** [41], která umožňuje vytvářet interaktivní webové aplikace v jazyce **Python**. Soubory se třídami tvořícími toto grafické uživatelské rozhraní se nacházejí ve složce `src/gui/` a diagram těchto tříd je zobrazen na obrázku D.1 v příloze D.

Každý režim vyhodnocení má svoji vlastní třídu, která se stará o přebírání dat z databáze a jejich zobrazení v uživatelském rozhraní. Třída **BaseEvaluator** zajišťuje společné funkce pro obě fáze vyhodnocení, jako například přebírání společných záznamů z databáze. Třídy **ProductionEvaluator** a **BenchmarkEvaluator** implementují vlastní metody pro každý režim vyhodnocení. Jelikož knihovna **streamlit** spouští celý kód znovu při každé interakci uživatele, **režim nejde změnit při běhu aplikace**, ale je potřeba znovu spustit **Textractor Studio** s nastaveným režimem, který se určuje podle proměnné prostředí (v souboru `.env`) pomocí proměnné `MODE`, která může nabývat hodnoty „production“ pro produkční režim nebo „benchmark“ pro evaluační režim. Bez zadání tohoto parametru se spustí evaluační režim.

6.5.1 Rozložení GUI

Uživatelské rozhraní je rozděleno na boční panel (vlevo) a hlavní obsah (vpravo). Náčrt rozložení grafického uživatelského rozhraní je zobrazen na obrázku 6.5.

Texttractor Studio

Enter your name:

Select document:
 ▼

☐ Phase 1 (Extraction)
☐ Phase 2 (Formalization)

Progress

Correct Incorrect

Obrázek 6.5: Náčrt rozložení grafického uživatelského rozhraní Texttractor Studio

6.5.1.1 Boční panel

V bočním panelu je pro přehlednost hodnotitelů **nutné** zadat jméno (tomu odpovídá atribut `evaluator_name` v databázových tabulkách `EXTRACTED_TEXT_EVAL` a `ANALYZED_TEXT_EVAL`). Dále je nutné vybrat, **který dokument se bude hodnotit**. V produkčním režimu se zobrazují všechny dokumenty, zatímco v evaluačním režimu se zobrazují pouze dokumenty, pro které existuje zlatý dataset.

Přepínačem ve spodní části bočního panelu je nutné zvolit, kterou fázi bude uživatel vyhodnocovat, tedy extrakční nebo formalizační.

6.5.1.2 Hlavní obsah

V hlavním obsahu se zobrazuje seznam záznamů z databáze, které **odpovídají zvolenému dokumentu a fázi**. Oba režimy mají různé rozložení pro obě fáze, jelikož výstupem obou režimů je jiný typ vyhodnocení.

V **produkčním režimu fáze 1** se zobrazuje zkopírovaný text, který jazykový model identifikoval jako požadavek, a vedle něj se nachází náhled dokumentu, ze kterého byla data extrahována (pomocí knihovny `streamlit_pdf_viewer` [42]). Uživatel

musí zkontrolovat, zda se tento zkopírovaný text skutečně v dokumentu nachází a označit ho jako správný nebo nesprávný.

V **produkčním režimu fáze 2** se zobrazuje postupně každý zkopírovaný text, který byl označen jako správný v extrakční fázi (tedy se v dokumentu skutečně nachází). Vedle něj se zobrazuje jeho formalizace, u které musí uživatel zkontrolovat správnost a označit ji jako správnou nebo nesprávnou.

V **evaluačním režimu fáze 1** se zobrazuje každý zkopírovaný text, který jazykový model identifikoval jako požadavek, a vedle něj se zobrazuje seznam zlatých požadavků z ručně vytvořeného zlatého datasetu. Hodnotitel musí vybrat a „propojit“ zkopírovaný text s některým ze zlatých požadavků (pomocí atributu `matched_golden_id`) a označit, že se shodují. Pokud hodnotitel zjistí, že se ve zlatém datasetu tento požadavek nevyskytuje, označí ho jako nesprávný.

Evaluační režim fáze 2 zobrazuje **ve formě karet**¹ každý formalizovaný požadavek z jazykového modelu (vlevo) a vedle něj (vpravo) zobrazuje formalizovaný požadavek ze zlatého datasetu. Uživatel musí porovnat oba formalizované požadavky a označit, zda se shodují, nebo ne. Může se stát, že jazykový model požadavek formalizoval „do více karet“², proto je potřeba porovnat všechny karty. V takovém případě je možné karty ručně spárovat – to je umožněno jejich vizuálním označením. Toto označení je zobrazeno pouze na úrovni uživatelského rozhraní a nemá žádný vliv na data uložená v databázi.

6.6 Logování

Aplikace využívá standardní knihovnu `logging` jazyka **Python**, obalenou do pomocné třídy `Log` (v souboru `src/utis.py`) a doplněnou o mechanismus potlačení výstupu do konzole při zobrazení průběhového ukazatele (knihovnou `tqdm` [43]).

Třída `Log` obsahuje čtyři statické metody odpovídající úrovním závažnosti:

- `debug(message: str, source=None)` – podrobné diagnostické informace, které se zapisují jen do souboru,
- `info(message: str, source=None)` – běžné provozní zprávy (zahájení extrakce nebo formalizace, výsledky operací),
- `warning(message: str, source=None)` – neočekávané, ale zvládnutelné situace,
- `error(message: str, source=None)` – chyby, po nichž zpracování daného záznamu přeskóčí.

¹Struktura karty odpovídá struktuře třídy `ParsedRequirement` ze schématu ve výpisu 6.5.

²Stále se jedná o formalizaci jednoho požadavku.

Každá tato metoda předá zprávu metodě `_log(level, message, source)` třídy `Log` (ta volá standardní logging systém), která z parametru `source` odvodí název komponenty, ze které log pochází. Pokud je `source` instancí třídy, použije se její název (například `ExtractionPhase`), jinak se použije řetězec „root“. Tím je každá zpráva v logu automaticky přiřazena ke komponentě, která ji vyvolala.

6.6.1 Inicializace

Metoda `setup_logging(log_to_file: bool = True)` třídy `Log` se volá jednou při startu aplikace a konfiguruje globální logovací systém prostřednictvím metody `logging.basicConfig()`.

Hlavní logger je nastaven na úroveň `DEBUG`, aby vypisoval veškeré zprávy. Filtrování probíhá až na úrovni handlerů:

- **StreamHandler (konzole)** – zobrazuje zprávy úrovně `INFO` a výše. Ladicí zprávy (`DEBUG`) se do konzole nevypisují.
- **FileHandler (soubor)** – zapisuje zprávy úrovně `DEBUG` a výše. Soubor nese název ve formátu `DD_MM_YYYY_HH_MM_SS.log` a ukládá se do adresáře `logs/`.

Formát každé zprávy je jednotný:

```
[HH:MM:SS:mmm] [ÚROVEŇ] [Komponenta] Text zprávy
```

Příklad záznamu logu o extrakci dokumentu může vypadat například takto:

```
[14:33:51:277] [INFO] [ExtractionPhase] Extracting 'thesis.pdf' (doc_id=3)
...
```

6.6.2 Průběhový ukazatel

Průběhový ukazatel knihovny `tqdm` a souběžný výpis logů do konzole by se navzájem rušily – každý řádek logu by rozbil vykreslení ukazatele. Proto je při každém zobrazení průběhového ukazatele konzolový výstup dočasně potlačen.

Tabulka 6.1 shrnuje, které úrovně zpráv se zobrazují v konzoli a které se zapisují do logovacího souboru. Zprávy úrovně `DEBUG` jsou určeny pro podrobné sledování průběhu a diagnostiku, ale nejsou zobrazeny v konzoli, aby nerušily uživatele. Zprávy úrovně `INFO` a výše jsou zobrazeny v konzoli i zapsány do souboru, protože obsahují klíčové informace o průběhu zpracování a případných problémech.

Tabulka 6.1: Výstupy zpráv jednotlivých úrovní

Úroveň	Konzole	Soubor
DEBUG	ne	ano
INFO	ano	ano
WARNING	ano	ano
ERROR	ano	ano

Vyhodnocení a testování

7

V této kapitole je popsána analýza výsledků z extrakční a formalizační fáze, které byly vytvořeny rozhraním Textractor Studio. Obsahuje různé metriky, které byly vypočteny pro otestování schopnosti různých velkých jazykových modelů extrahovat data z dlouhých textů. **Prezentované výsledky jsou výsledkem analýzy výstupů pouze evaluační fáze rozhraní Textractor Studio.**

7.1 Datový korpus

Pro potřeby tohoto vyhodnocení byly ručně analyzovány dva dokumenty typu PDF – specifikace softwarových požadavků:

1. **WebGL Card Game Platform:** Specifikace požadavků na platformu karet-ních her založenou na webovém rozhraní, vytvořenou pomocí technologie WebGL [32]. Tento dokument je označován jako *Hra*.
2. **Nemocniční informační systém:** Technická specifikace dodávky nového nemocničního informačního systému pro Vojenskou nemocnici Brno [33]. Tento dokument je označován jako *NIS*.

7.1.1 Parametry testovacích dokumentů

Při výběru testovacích dat byl kladen důraz na to, aby jeden dokument byl delší a jeden kratší. Zároveň je důležité testování na dokumentech psaných v českém i anglickém jazyce, protože jazykové modely byly vyzvány k přizpůsobení se danému jazyku. Potřebné parametry těchto dokumentů jsou zobrazeny v tabulce 7.1.

Pro oba testovací dokumenty proběhla ruční analýza všech požadavků a byl pro ně vytvořen zlatý dataset.

Tabulka 7.1: Parametry testovacích dokumentů

	Hra	NIS
Jazyk	Anglický	Český
Počet stran	28	106
Počet ručně identifikovaných požadavků	107	984

7.2 Testovací parametry

V této sekci jsou popsány vybrané modely a počítání cen extrakce a formalizace. Systémová zpráva, která definuje roli pro jazykový model a příkazy pro extrakci je obsažena v příloze F. Pro formalizaci je zpráva vypsána v příloze G.

7.2.1 Vybrané modely

Jelikož Textractor funguje ve dvou fázích, pro extrakci i formalizaci požadavků **je možné zvolit jiný model**. Tabulka 7.2 ukazuje použité modely v obou těchto fázích.

Byly použité modely od společnosti OpenAI, konkrétně gpt-5.4-nano [39] a od společnosti Mistral (model mistral-large-latest [40] – konkrétně Mistral 3 Large). Pro kratší dokument byl také vyzkoušen ve formalizační fázi lokální model llama3.2:latest [20] od společnosti Meta. Tento model nebyl spouštěn v extrakční fázi ani pro delší dokument NIS z důvodu časové náročnosti běhu. Model mistral-large-latest nebyl schopen vyextrahovat dokument NIS. V odpovědi API požadavku na extrakci tohoto dokumentu se nacházela i po opakování pokusu chyba HTTP 503 a zpráva „Internal server error“ s hodnotou parametru type „unreachable_backend“.

Při prvních testech programu Textractor byl vyzkoušen také model gpt-4o [44] od společnosti OpenAI, který je vhodný pro analýzu dlouhých textů [16]. Jenže díky jeho maximálnímu počtu výstupních tokenů (16 384) nebylo možné v extrakční fázi zpracovat **celý** dokument NIS – model vrátil jen část do obsažení limitu, proto tento model nebyl do hodnocení zahrnut.

7.2.2 Ceny

Ve vyhodnocení se objevuje porovnání cen a extrakcí. Společnosti účtují využití jazykových modelů přes API pomocí dvou ukazatelů – vstupních tokenů a výstupních tokenů. Tyto ceny se uvádí za 1 milion tokenů v amerických dolarech. Tabulka 7.3 ukazuje ceny pro použité modely podle ceníků na webových stránkách společností (pro OpenAI [45], pro Mistral [46]) k datu spuštění extrakce a formalizace, tedy 3. 4. 2026.

Tabulka 7.2: Použité jazykové modely pro vyhodnocení pro dokumenty podle fází

Dokument	Modely	
	Fáze 1	Fáze 2
Hra	gpt-5.4-nano, mistral-large-latest	gpt-5.4-nano, mistral-large-latest, llama3.2:latest
NIS	gpt-5.4-nano	gpt-5.4-nano, mistral-large-latest

Tabulka 7.3: Ceny vstupních a výstupních tokenů pro LLM

Model	Vstupní tokeny	Výstupní tokeny
gpt-5.4-nano	0,2 \$ / 1M tokenů	1,25 \$ / 1 M tokenů
mistral-large-latest	0,5 \$ / 1 M tokenů	1,5 \$ / 1 M tokenů

Model llama3.2:latest v tabulce chybí, jelikož byl spuštěn lokálně na osobním notebooku¹. Cena vstupních a výstupních tokenů u tohoto modelu je počítána jako 0 Kč.

Pro lepší přehlednost byly ceny převedeny na české koruny podle kurzu 20,608 Kč za 1 \$ (k datu 17. 4. 2026) [47].

7.3 Časy

V tabulce 7.4 jsou vypsané **průměrné ceny a časy extrakce jednoho dokumentu podle modelů**². Modelu mistral-large-latest trvalo několikanásobně větší čas extrahovat požadavky z kratšího dokumentu Hra, a to i s vyšší cenou.

V tabulce 7.5 jsou vypsané **průměrné ceny a časy formalizace jednoho požadavku podle modelů**. Ve formalizační fázi model gpt-5.4-nano také dominuje v průměrné rychlosti formalizace na jeden požadavek. V dokumentu Hra je však o 0,14 s rychlejší než u dokumentu NIS, což je způsobeno tím, že v tomto dokumentu byly celkově extrahovány kratší věty. Časově nejhorší je však model llama3.2:latest, kterému trvalo průměrně jeden požadavek v dokumentu Hra extrahovat za 9,74 sekundy.

¹Notebook s procesorem Intel Core i5 1335U Raptor Lake a integrovanou grafickou kartou Intel Iris Xe Graphics.

²Jak bylo zmíněno v sekci 7.2.1, model mistral-large-latest nebyl schopen zpracovat dokument NIS.

Tabulka 7.4: Průměrné ceny a časy extrakce za dokument podle modelů

Dokument	Model	Průměrný čas extrakce	Průměrná cena extrakce
NIS	gpt-5.4-nano	222,61 s	1,30 Kč
Hra	gpt-5.4-nano	18,62 s	0,11 Kč
	mistral-large-latest	75,37 s	0,19 Kč

Tabulka 7.5: Průměrné ceny a časy formalizace za požadavek podle modelů

Dokument	Model	Průměrný čas	Průměrná cena
NIS	gpt-5.4-nano	1,49 s	0,0057 Kč
	mistral-large-latest	5,43 s	0,0379 Kč
Hra	gpt-5.4-nano	1,35 s	0,0047 Kč
	llama3.2:latest	9,74 s	0 Kč
	mistral-large-latest	4,26 s	0,0308 Kč

Celkový čas ruční extrakce a formalizace těchto dokumentů (tedy tvorba zlatého datasetu) trval pro dokument Hra přibližně 2 hodiny a pro dokument NIS přibližně 14 hodin a 30 minut.

7.4 Kvalita

Kvalita modelů byla také měřena podle fází. V extrakční fázi byl pro každý LLM vypočítán recall (vzorec 3.4), což označuje, **kolik procent ze všech reálně existujících požadavků (to, co našel člověk ve zlatém datasetu) model dokázal najít**. Dále byla použita metrika precision (vzorec 3.5), která ukazuje, **kolik procent z toho, co model celkem vyprodukoval, bylo skutečně správně**. Posledním ukazatelem je F1 skóre (vzorec 3.6), které spojí tyto metriky dohromady a vytvoří celkové hodnocení výkonu harmonickým průměrem hodnot recall a precision.

Ve formalizační fázi byla spočítána metrika accuracy (sekce 3.5.3, vzorec 3.7), která označuje, **kolik procent z hodnocených formalizací model zvládl vygenerovat správně (to jsou formalizace, ve kterých nic nechybělo a nic chybného nepřebývalo)**. Formalizace vyprodukované modelem byly prováděny jen nad zlatými záznamy z extrakční fáze (fáze 1).

Tabulka 7.6: Evaluační metriky v extrakční fázi podle modelů a dokumentů

Metrika	NIS		Hra	
	gpt-5.4-nano	mistral-large-latest	gpt-5.4-nano	mistral-large-latest
TP	313	—	65	68
FP	838	—	231	295
RP	984	—	107	107
Precision	27,19 %	—	21,96 %	18,73 %
Recall	31,81 %	—	60,75 %	63,55 %
F1 skóre	29,32 %	—	32,26 %	28,94 %

7.4.1 Extrakce

V extrakční fázi byly v rozhraní Textractor Studio ohodnoceny všechny provedené extrakce všemi použitými modely pro dané dokumenty. Tabulka 7.6 ukazuje F1 skóre, precision a recall podle modelů pro oba dokumenty v extrakční fázi. Jsou v ní zobrazeny také hodnoty, podle kterých se tyto metriky vypočetly. Je vidět, že model `mistral-large-latest` nebyl schopný vyextrahovat dokument NIS. U dokumentu Hra má větší recall než `gpt-5.4-nano`, ale menší precision – díky tomu má menší F1 skóre.

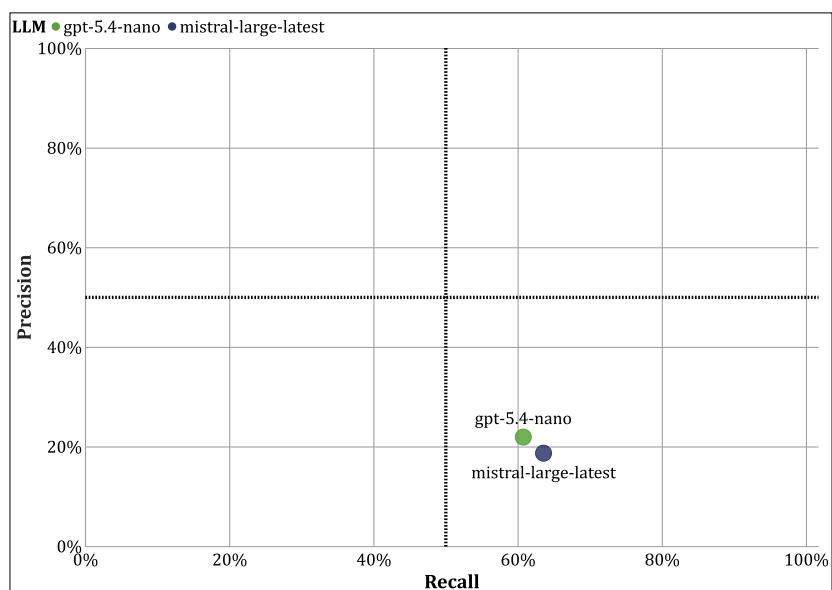
Porovnání recall a precision u těchto modelů³ ukazuje obrázek 7.1. Bodový graf na tomto obrázku je rozdělen na kvadranty, které určují úspěšnost. Nejlepší výsledky precision se objeví v horních kvadrantech a nejlepší výsledky recall v pravých kvadrantech. Oba tyto modely se nacházejí v pravém dolním kvadrantu, což znamená, že se jim podařilo najít více jak polovinu požadavků, ale více jak polovina jich byla nesprávná.

7.4.2 Formalizace

Tabulka 7.7 zobrazuje accuracy podle modelů pro oba dokumenty ve formalizační fázi. V této fázi se modelům dařilo výrazně lépe formalizovat požadavky (fáze 2) než je extrahovat (fáze 1), kromě modelu `llama3.2:latest`, který má ze všech modelů nejmenší accuracy.

Čísla v závorce za dokumenty označují **počet správných formalizací z počtu ohodnocených formalizací**. Z důvodu velké časové náročnosti nebyly v rozhraní Textractor Studio vyhodnoceny všechny vygenerované formalizace u dokumentu

³Pouze pro dokument Hra, jelikož tento dokument úspěšně prošel extrakční fází oběma modely.



Obrázek 7.1: Bodový graf zobrazující recall a precision u dokumentu Hra

Tabulka 7.7: Accuracy ve formalizační fázi podle modelů

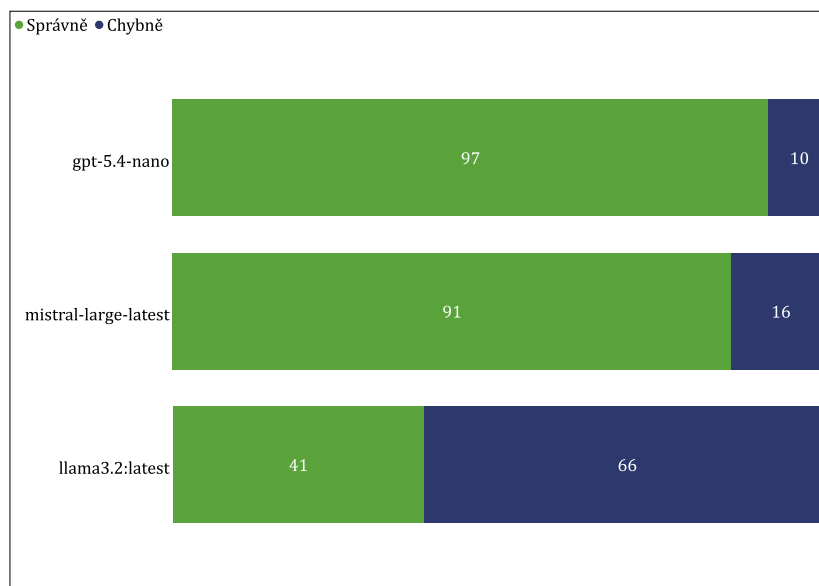
Dokument	Accuracy (správně/celkem)		
	gpt-5.4-nano	mistral-large-latest	llama3.2:latest
NIS	88,64 % (234/264)	87,83 % (231/263)	—
Hra	90,65 % (97/107)	85,05 % (91/107)	38,32 % (41/107)

NIS, což je třeba brát v potaz u jejich accuracy. Tabulka 7.8 ukazuje počty ohodnocených formalizací pro oba dokumenty a pro všechny použité modely. Pro dokument NIS bylo ohodnoceno sekvenčně 527 požadavků z celkem 1968.

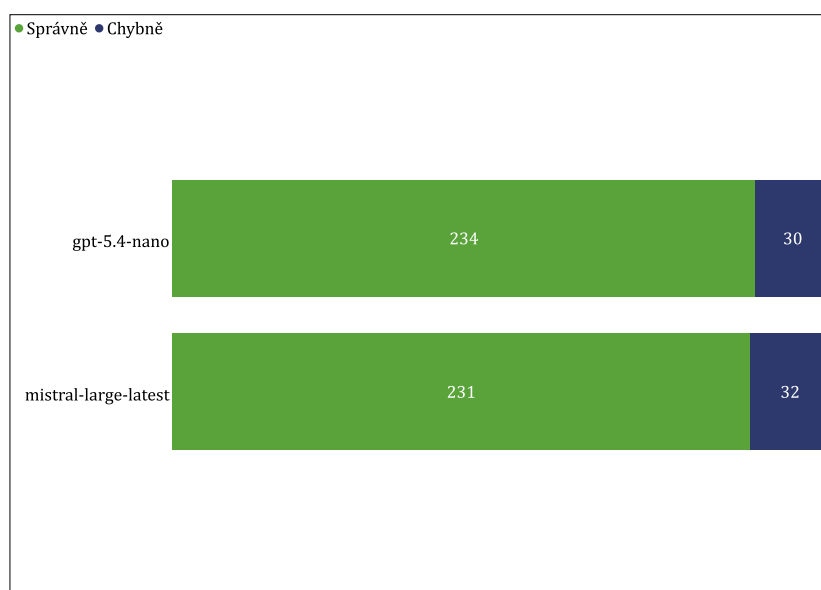
Obrázky 7.2 a 7.3 ukazují grafy s počty správně a chybně formalizovaných požadavků použitých modelů pro dokumenty Hra a NIS. Pro dokument NIS existuje více správně extrahovaných požadavků, což je způsobeno jeho velikostí.

Tabulka 7.8: Počty ohodnocených formalizací podle dokumentů a modelů

Dokument	Model	Celkem	Ohodnoceno
NIS	gpt-5.4-nano	984	264
	mistral-large-latest	984	263
	Total	1968	527
Hra	gpt-5.4-nano	107	107
	mistral-large-latest	107	107
	llama3.2:latest	107	107
	Celkem	321	321
Celkem	gpt-5.4-nano	1091	371
	mistral-large-latest	1091	370
	llama3.2:latest	107	107
	Celkem	2289	848



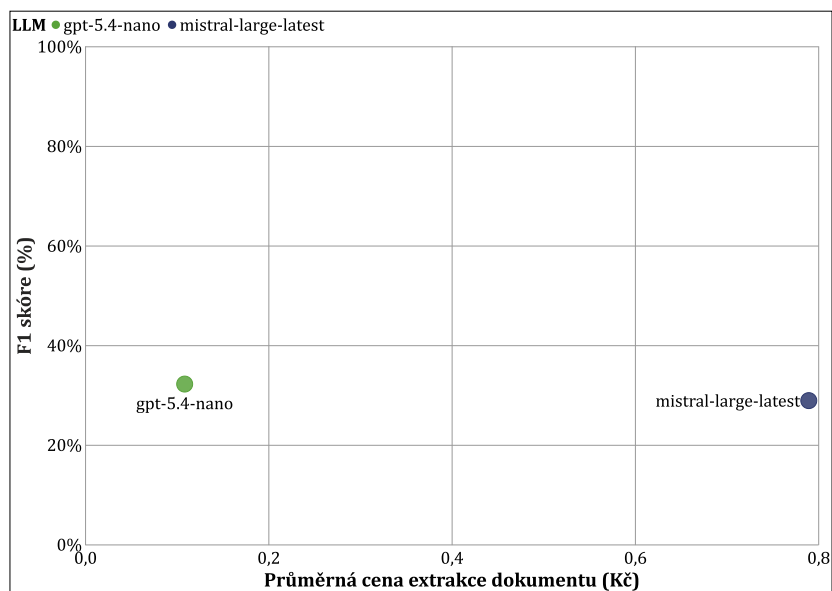
Obrázek 7.2: Počet správných a špatných formalizací podle modelů pro dokument Hra



Obrázek 7.3: Počet správných a špatných formalizací podle modelů pro dokument NIS

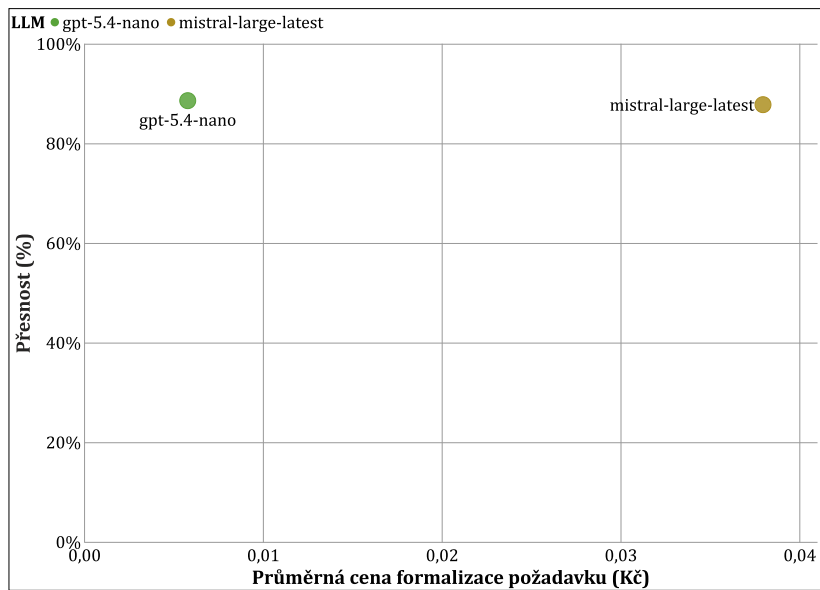
7.5 Poměr ceny a kvality

Obrázek 7.4 ukazuje v bodovém grafu poměr průměrné ceny extrakce dokumentu a kvality (pomocí F1 skóre).

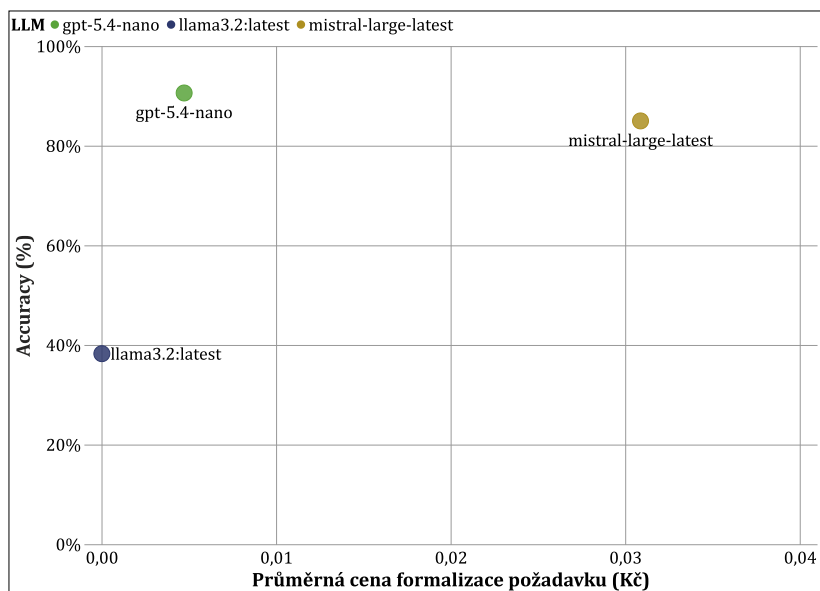


Obrázek 7.4: Bodový graf zobrazující poměr F1 skóre recall a průměrné ceny u dokumentu Hra

Na obrázku 7.5 je zobrazeno srovnání průměrné ceny formalizace jednoho požadavku a accuracy pro dokument NIS. Na obrázku 7.6 pro dokument Hra se stejným srovnáním přibyl model llama3.2:latest, který je sice levný, ale výrazně horší v accuracy oproti ostatním modelům.



Obrázek 7.5: Bodový graf zobrazující poměr accuracy a průměrné ceny u dokumentu NIS



Obrázek 7.6: Bodový graf zobrazující poměr accuracy a průměrné ceny u dokumentu Hra

7.6 Časté chyby modelů

Velké jazykové modely se dopouštěly při extrakci i formalizaci častých chyb, které byly pozorovány při hodnocení v rozhraní Textractor Studio. Tyto chyby jsou popsány v tabulce 7.9. První sloupec reprezentuje typ chyby, kde písmeno reprezentuje fázi, ve které se chyba vyskytovala – „E“ pro extrakční fázi a „F“ pro formalizační fázi. Číslo za daným písmenem znamená označení chyby.

Tabulka 7.9: Přehled častých chyb LLM u extrakční a formalizační fáze

	Popis chyby
E1	I přes výslovný zákaz v systémové zprávě LLM nedodávají kontext k požadavku (ani ve formě nadpisů, ani jinak). Toto poté způsobuje zneplatnění požadavku, protože z něj nelze pochopit širší kontext. Například věta „Možnost volby formátu A4 a A5.“ neříká, v jaké sekci systému se to má vyskytovat, nebo kde tento formát vybrat.
E2	Seznamy s odrážkami jsou rozdělovány jako různé požadavky. Výsledný text jednoho požadavku pak nepředává potřebnou informaci.
E3	Zkopírování části požadavku bez dodatečných informací, které do něj jasně patří, i přes to, že jsou součástí nějaké věty (často výrazy „...např. ...“ nebo „...na odkaze ...“).
E4	Zkopírování jen části souvětí.
E5	Zkopírování věty obsahující požadavek a ignorování dodatečných závorek pro upřesnění.
F6	Modely překládají text do anglického jazyka, i přes výslovný zákaz překládání textu do jiného jazyka v systémové zprávě.*
F7	Objevují se halucinace ve formalizovaných textech, zpravidla pokud model usoudí, že nemá dostatek kontextu pro doplnění.
F8	Model llama3.2:latest při některých požadavcích místo reálné formalizace použije přesný příklad uvedený v systémové zprávě.
F9	Model llama3.2:latest většinu textu některých požadavků neformalizuje, jen z nich přepíše klíčové slovo.

(tabulka pokračuje na další stránce)

Tabulka 7.9 (pokračování z předchozí stránky)

	Popis chyby
F10	Při formalizaci modely často ignorují webové odkazy.
F11	Pokud extrahovaný text obsahuje nadpis, modely ho většinou nerozpoznají a soustředí se na vlastnosti v požadavku, což v některých případech nevádí, pokud tyto vlastnosti obsahují dostatečný kontext a lze z nich pochopit účel požadavku.
F12	Model <code>mistral-large-latest</code> někdy identifikuje náhodná čísla jako kvantifikovatelná data, i když nesouvisí s požadavkem.
F13	Při formalizaci kvantifikovatelných dat LLM někdy nepřidá dodatečnou informaci k parametru jednotky. Například u informace „... minimálně 10 let.“ může být parametr jednotky jen „let“.*
F14	Modely si mohou splést uvedení příkladu („... např. ...“) v požadavku jako jedinou možnost, jak danou činnost vykonat, nikoliv jako příklad.

* Toto chování není bráno jako chyba při vyhodnocení v rozhraní Textactor Studio z důvodu velmi častého výskytu a nerelevanci k extrakci/formalizaci informací jinak obsažených v požadavku.

7.7 Shrnutí

Z informací v této kapitole vyšel nejlépe model gpt-5.4-nano, i přes to, že model mistral-large-latest disponuje vyšším recall u dokumentu Hra. Nedokázal ale v první fázi zpracovat dokument NIS, s čímž si naopak model gpt-5.4-nano poradit dokázal.

Pokud by ruční extrakci a formalizaci vykonával člověk, strávil by na tom o 959,1 minut déle, než model gpt-5.4-nano. Tabulka 7.10 ukazuje srovnání cen a časů práce člověka oproti práci tohoto modelu v extrakční i formalizační fázi v obou dokumentech. Pokud je tedy definováno ocenění ruční práce extrakce a formalizace člověka jako 250 Kč za hodinu, při použití tohoto modelu **se ušetří 4 117,59 Kč. Pro tyto výpočty byly použity přibližné časy ruční extrakce a formalizace popsány v sekci 7.3.**

Je ale důležité zmínit, že **požadované F1 skóre a accuracy nemusí být pro určité úkoly dostačující.**

Tabulka 7.10: Srovnání cen a časů práce člověka proti práci modelu gpt-5.4-nano

Celková cena člověka	4 125,00 Kč
Celková cena modelu	7,41 Kč
Celkový čas modelu	30,90 min
Celkový čas člověka	990 min
Ušetřený čas díky modelu	959,10 min
Ušetřené peníze díky modelu	4 117,59 Kč

Tato práce prozkoumala, jak kvalitně dokážou velké jazykové modely extrahovat a formalizovat data ze specifikací softwarových požadavků. Nejlepší z porovnávaných modelů podle kvality a ceny je gpt-5.4-nano, který dokázal zpracovat oba testované dokumenty. Další vyzkoušené modely byly mistral-large-latest pro obě fáze a llama3.2:latest pro formalizační fázi.

V extrakční fázi dosáhl model gpt-5.4-nano F1 skóre 29,32 % pro delší dokument a 32,26 % pro kratší dokument. Pro srovnání, model mistral-large-latest dosáhl F1 skóre 28,94 % na kratším dokumentu a delší nezpracoval. Tyto hodnoty byly ověřeny ručně oproti referenčním extrakcím a jsou pro produkční nasazení nedostatečné. Modely totiž často nedodávaly kontext nezbytný pro pochopení požadavku (například nadpis nebo okolní věty).

Ve formalizační fázi, která byla prováděna na jednotlivých extrahovaných požadavcích, dosáhly modely výrazně lepších výsledků. Accuracy modelu gpt-5.4-nano se pohybovala mezi 88,64 % a 90,65 %, mistral-large-latest mezi 85,05 % a 87,83 % a u llama3.2:latest dosahovala pouze 38,32 %. Může být tedy vhodné některé LLM pro formalizaci využít, ale menší lokální modely se na tento úkol nehodí. Je však dobré zkontrolovat výstupy, například v rozhraní Textractor Studio. Častou chybou modelů bylo domýšlení informací.

Vyvinutý nástroj je připraven pro budoucí rozšíření a další testování na různých dokumentech, nejen SRS. Výstupy jazykových modelů úzce souvisí se systémovou zprávou, která modelu předává instrukce pro daný úkol. Tuto zprávu je možné ladit pro určitý typ dokumentů a pozorovat, jak se výsledky podle těchto zpráv mění. Další možnost pro zlepšení výsledků může být rozdělení dokumentu a zpracování po menších částech, což by vyřešilo problém neschopnosti některých modelů (například mistral-large-latest) zpracovat příliš dlouhý text.

Grafické uživatelské rozhraní Textractor Studio bylo testováno pro evaluační režim, který používá ručně extrahované a formalizované požadavky. Další práce se může také zaměřit na vylepšení produkčního režimu, který nepracuje s referenčními informacemi, ale pouze s původními dokumenty.

Seznam obrázků

5.1	Postup běhu nástroje Textractor	23
5.2	Rozdělení režimů a fází v rozhraní Textractor Studio	26
6.1	Diagram tříd pro načítání dokumentů z disku	33
6.2	Diagram tříd sloužící pro uložení dokumentů do databáze	33
6.3	Diagram tříd pro fáze zpracování dokumentů jazykovým modelem . .	36
6.4	Diagram tříd sloužících pro komunikaci s jazykovými modely	37
6.5	Náčrt rozložení grafického uživatelského rozhraní Textractor Studio .	40
7.1	Bodový graf zobrazující recall a precision u dokumentu Hra	50
7.2	Počet správných a špatných formalizací podle modelů pro dokument Hra	51
7.3	Počet správných a špatných formalizací podle modelů pro dokument NIS	52
7.4	Bodový graf zobrazující poměr F1 skóre recall a průměrné ceny u do- kumentu Hra	53
7.5	Bodový graf zobrazující poměr accuracy a průměrné ceny u dokumentu NIS	54
7.6	Bodový graf zobrazující poměr accuracy a průměrné ceny u dokumentu Hra	54
A.1	ERA model databáze pro extrakci dat z projektové dokumentace . . .	72
C.1	Diagram tříd databázových repozitářů	76
D.1	Diagram tříd pro grafické uživatelské rozhraní Textractor Studio . . .	78

Seznam tabulek

5.1	Význam hodnocení v rozhraní Textractor Studio	27
6.1	Výstupy zpráv jednotlivých úrovní	43
7.1	Parametry testovacích dokumentů	46
7.2	Použité jazykové modely pro vyhodnocení pro dokumenty podle fází .	47
7.3	Ceny vstupních a výstupních tokenů pro LLM	47
7.4	Průměrné ceny a časy extrakce za dokument podle modelů	48
7.5	Průměrné ceny a časy formalizace za požadavek podle modelů	48
7.6	Evaluační metriky v extrakční fázi podle modelů a dokumentů	49
7.7	Accuracy ve formalizační fázi podle modelů	50
7.8	Počty ohodnocených formalizací podle dokumentů a modelů	51
7.9	Přehled častých chyb LLM u extrakční a formalizační fáze	55
7.10	Srovnání cen a časů práce člověka proti práci modelu gpt-5.4-nano .	57
B.1	Základní struktura složek a souborů programu Textractor	73

Seznam výpisů

6.1	Definice pydantic schématu zlatého datasetu	34
6.2	Definice pydantic schématu zlatého parametru	34
6.3	Příklad zlatého požadavku ve formátu JSON	35
6.4	Definice schématu pro extrakční fázi	38
6.5	Definice schématu pro formalizační fázi	39

Bibliografie

1. COWIE, Jim; LEHNERT, Wendy. Information extraction. *Commun. ACM*. 1996, roč. 39, č. 1, s. 80–91. ISSN 0001-0782. Dostupné z DOI: 10.1145/234173.234209.
2. CHEN, Linjie et al. A Method for Extracting Information from Long Documents that Combines Large Language Models with Natural Language Understanding Techniques. In: *2023 4th International Conference on Computer, Big Data and Artificial Intelligence (ICCBD+AI)*. 2023, s. 43–47. Dostupné z DOI: 10.1109/ICCBD-AI62252.2023.00015.
3. WANG, Junmiao. A Study of The OCR Development History and Directions of Development. *Highlights in Science, Engineering and Technology*. 2023, roč. 72, s. 409–415. Dostupné z DOI: 10.54097/bm665j77.
4. LI, Yunyao; KRISHNAMURTHY, Rajasekar; RAGHAVAN, Sriram; VAITHYANATHAN, Shivakumar; JAGADISH, H. V. Regular expression learning for information extraction. In: *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. Honolulu, Hawaii: Association for Computational Linguistics, 2008, s. 21–30. EMNLP '08.
5. LARKIN, Henry. Object oriented regular expressions. In: *2008 8th IEEE International Conference on Computer and Information Technology*. 2008, s. 491–496. Dostupné z DOI: 10.1109/CIT.2008.4594724.
6. G, Veena; R, Hemanth; HAREESH, Jithin. Relation Extraction in Clinical Text using NLP Based Regular Expressions. In: *2019 2nd International Conference on Intelligent Computing, Instrumentation and Control Technologies (ICICT)*. 2019, sv. 1, s. 1278–1282. Dostupné z DOI: 10.1109/ICICT46008.2019.8993274.
7. LI, Hang. Language models: past, present, and future. *Commun. ACM*. 2022, roč. 65, č. 7, s. 56–63. ISSN 0001-0782. Dostupné z DOI: 10.1145/3490443.
8. HAYES, Brian. First Links in the Markov Chain. *American Scientist*. 2013, roč. 101, s. 92–97. Dostupné z DOI: 10.1511/2013.101.92.
9. MAJUMDER, P; MITRA, M; CHAUDHURI, BB. N-gram: a language independent approach to IR and NLP. In: *International conference on universal knowledge and language*. 2002, sv. 2.

10. JOHN LAWLER. *The ChomskyBot* [online]. 1998-09-25. [cit. 2025-11-30]. Dostupné z: <http://www.chomskybot.com/cgi-bin/chomsky.pl>.
11. WIKIPEDIE. *Noam Chomsky — Wikipedie: Otevřená encyklopedie* [online]. 2025. [cit. 2025-11-30]. Dostupné z: https://cs.wikipedia.org/w/index.php?title=Noam_Chomsky&oldid=25652333.
12. BENGIO, Yoshua; DUCHARME, Réjean; VINCENT, Pascal; JAUVIN, Christian. A neural probabilistic language model. *Journal of machine learning research*. 2003, roč. 3, č. Feb, s. 1137–1155.
13. VASWANI, Ashish et al. Attention Is All You Need. *CoRR*. 2017, roč. abs/1706.03762. Dostupné z arXiv: 1706.03762.
14. LUITSE, Dieuwertje; DENKENA, Wiebke. The great Transformer: Examining the role of large language models in the political economy of AI. *Big Data & Society*. 2021, roč. 8, č. 2, s. 20539517211047734. Dostupné z DOI: 10.1177/20539517211047734.
15. KASNECI, Enkelejda et al. ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences*. 2023, roč. 103, s. 102274. ISSN 1041-6080. Dostupné z DOI: <https://doi.org/10.1016/j.lindif.2023.102274>.
16. OPENAI. *Models* [online]. [cit. 2025-11-30]. Dostupné z: <https://platform.openai.com/docs/models>.
17. OPENAI. *Migrate to the Responses API* [online]. [cit. 2026-01-19]. Dostupné z: <https://platform.openai.com/docs/guides/migrate-to-responses>.
18. OPENAI. *Structured model outputs* [online]. [cit. 2026-01-19]. Dostupné z: <https://developers.openai.com/api/docs/guides/structured-outputs>.
19. PYDANTIC. *pydantic* [online]. 2025. Ver. 2.12.5 [cit. 2026-03-28]. Dostupné z: <https://github.com/pydantic/pydantic/>.
20. OLLAMA. *llama3.2:latest* [online]. [B.r.]. [cit. 2026-04-03]. Dostupné z: <https://ollama.com/library/llama3.2:latest>.
21. OLLAMA. *Structured Outputs* [online]. [cit. 2026-01-19]. Dostupné z: <https://docs.ollama.com/capabilities/structured-outputs>.
22. MISTRAL AI. *Chat Endpoints* [online]. [cit. 2026-01-19]. Dostupné z: <https://docs.mistral.ai/api>.
23. MISTRAL AI. *Sampling* [online]. [cit. 2026-01-19]. Dostupné z: <https://docs.mistral.ai/models/best-practices/sampling>.
24. ANTHROPIC. *Context windows* [online]. [cit. 2026-01-27]. Dostupné z: <https://platform.claude.com/docs/en/build-with-claude/context-windows>.

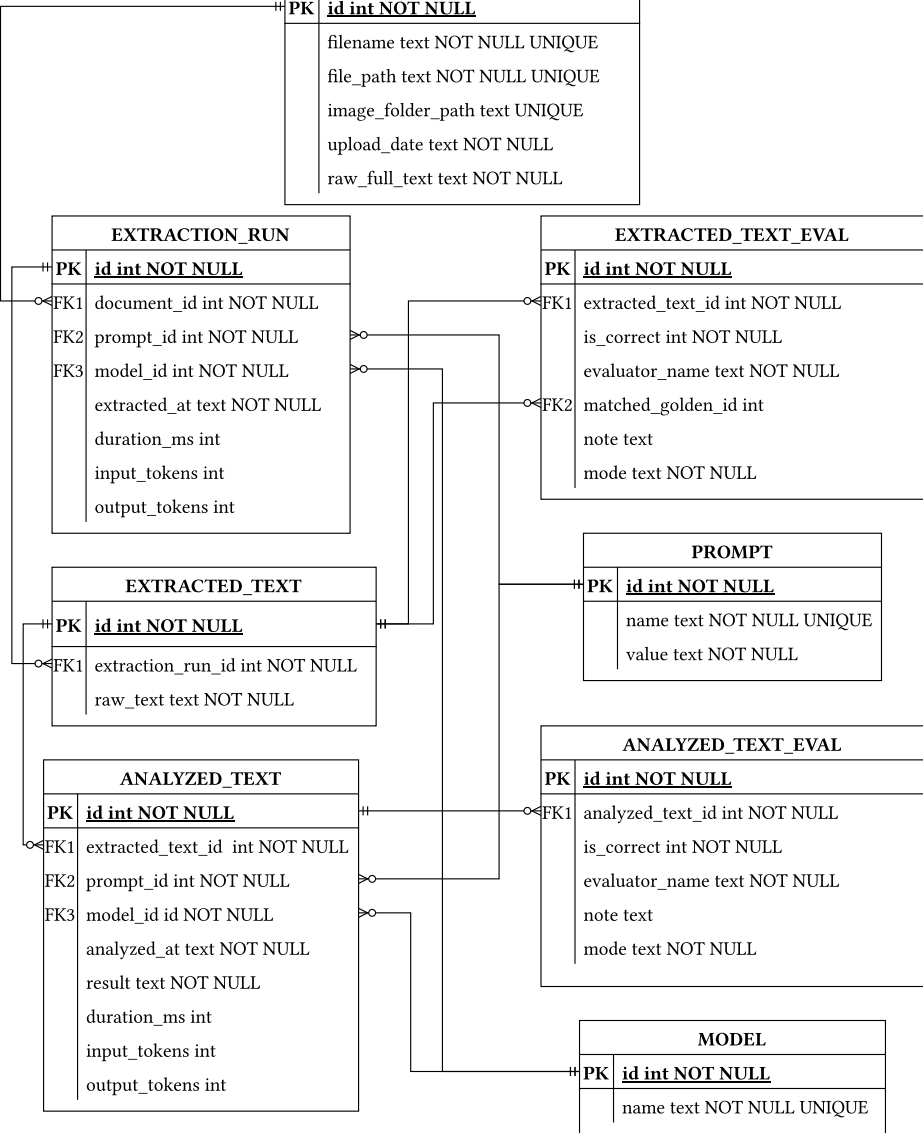
25. HAGENDORFF, Thilo; FABI, Sarah; KOSINSKI, Michal. Human-like intuitive behavior and reasoning biases emerged in large language models but disappeared in ChatGPT. *Nature Computational Science*. 2023, roč. 3, č. 10, s. 833–838.
26. CHEN, Yanda et al. *Reasoning Models Don't Always Say What They Think*. 2025. Dostupné z arXiv: 2505.05410 [cs.CL].
27. OPENAI. *Text generation* [online]. [cit. 2025-11-30]. Dostupné z: <https://platform.openai.com/docs/guides/text>.
28. BARTELS, Stefan; CARUS, Jasmin. From text to data: Open-source large language models in extracting cancer related medical attributes from German pathology reports. *International Journal of Medical Informatics*. 2025, roč. 203, s. 106022. ISSN 1386-5056. Dostupné z DOI: <https://doi.org/10.1016/j.ijmedinf.2025.106022>.
29. ARTETXE, Mikel et al. *Efficient Large Scale Language Modeling with Mixtures of Experts*. 2022. Dostupné z arXiv: 2112.10684 [cs.CL].
30. LI, Xinze; CAO, Yixin; MA, Yubo; SUN, Aixin. *Long Context vs. RAG for LLMs: An Evaluation and Revisits*. 2024. Dostupné z arXiv: 2501.01880 [cs.CL].
31. POWERS, David; AILAB. Evaluation: From precision, recall and F-measure to ROC, informedness, markedness & correlation. *J. Mach. Learn. Technol.* 2011, roč. 2, s. 2229–3981. Dostupné z DOI: 10.9735/2229–3981.
32. SARA FARAG. *Software Requirements Specification WebGL Card Game Platform* [online]. Bellevue College, 2019-03-15. Ver. 1.3 [cit. 2026-04-03]. Dostupné z: https://www.bellevuecollege.edu/wp-content/uploads/sites/135/2019/04/SRS_WebGLGame.pdf.
33. VOJENSKÁ NEMOCNICE BRNO. *VN Brno – nemocniční informační systém* [online]. 2024-10-22. [cit. 2026-04-03]. Specifikace požadavků. Vojenská nemocnice Brno. Dostupné z: <https://nen.nipez.cz/Soubor.aspx?id=2471203647&typ=.zip&velikost=7999735B>.
34. PYLINT CONTRIBUTORS. *PyLint* [online]. 2026. Ver. 4.0.5 [cit. 2026-03-28]. Dostupné z: <https://github.com/pylint-dev/pylint>.
35. SQLITE. *SQLite* [online]. [B.r.]. Ver. 3.50.2 [cit. 2026-03-28]. Dostupné z: <https://sqlite.org/>.
36. MICROSOFT CORPORATION. *Microsoft Power BI Desktop* [online]. 2026. Ver. 2.140.0 [cit. 2026-04-03]. Dostupné z: <https://powerbi.microsoft.com/desktop/>.
37. SINGER-VINE, Jeremy; THE PDFPLUMBER CONTRIBUTORS. *pdfplumber*. 2026. Ver. 0.11.9. Dostupné také z: <https://github.com/jsvine/pdfplumber>.

38. PYPDFIUM2-TEAM. *pypdfium2* [online]. 2026. Ver. 5.6.0 [cit. 2026-03-28]. Dostupné z: <https://github.com/pypdfium2-team/pypdfium2>.
39. OPENAI. *GPT-5.4 nano* [online]. 2026. Gpt-5.4-nano-2026-03-17 [cit. 2026-04-03]. Dostupné z: <https://developers.openai.com/api/docs/models/gpt-5.4-nano>.
40. MISTRAL AI. *Mistral Large 3* [online]. 2025. Mistral-large-2512 [cit. 2026-04-03]. Dostupné z: <https://docs.mistral.ai/models/model-cards/mistral-large-3-25-12>.
41. SNOWFLAKE. *Streamlit* [online]. 2026. Ver. 1.55.0 [cit. 2026-03-28]. Dostupné z: <https://streamlit.io/>.
42. SCIENCIALAB. *streamlit-pdf-viewer* [online]. 2026. Ver. 0.0.28 [cit. 2026-03-28]. Dostupné z: <https://github.com/ScienciaLAB/streamlit-pdf-viewer>.
43. TQDM. *tqdm* [online]. 2026. Ver. 4.67.3 [cit. 2026-03-28]. Dostupné z: <https://github.com/tqdm/tqdm>.
44. OPENAI. *GPT-4o* [online]. 2024. Gpt-4o-2024-11-20 [cit. 2026-04-03]. Dostupné z: <https://developers.openai.com/api/docs/models/gpt-4o>.
45. OPENAI. *API Pricing* [online]. [cit. 2026-04-17]. Dostupné z: <https://openai.com/api/pricing/>.
46. MISTRAL AI. *Pricing* [online]. [cit. 2026-04-17]. Dostupné z: <https://mistral.ai/pricing/#api>.
47. KURZY.CZ; ALIAWEB. *USD, americký dolar - převod měn na CZK, českou korunu* [online]. [cit. 2026-04-17]. Dostupné z: <https://www.kurzy.cz/kurzy-men/prevodnik-men/USD-CZK/>.
48. MITMPROXY. *pdoc* [online]. 2025. Ver. 16.0.0 [cit. 2026-05-03]. Dostupné z: <https://github.com/mitmproxy/pdoc>.

ERA model databáze



Následující stránka obsahuje obrázek A.1 zobrazující ERA model databáze, kterou využívá program Textractor.



Obrázek A.1: ERA model databáze pro extrakci dat z projektové dokumentace

Souborová struktura



Tabulka B.1 zobrazuje strukturu složek a spouštěcích skriptů programu Textractor.

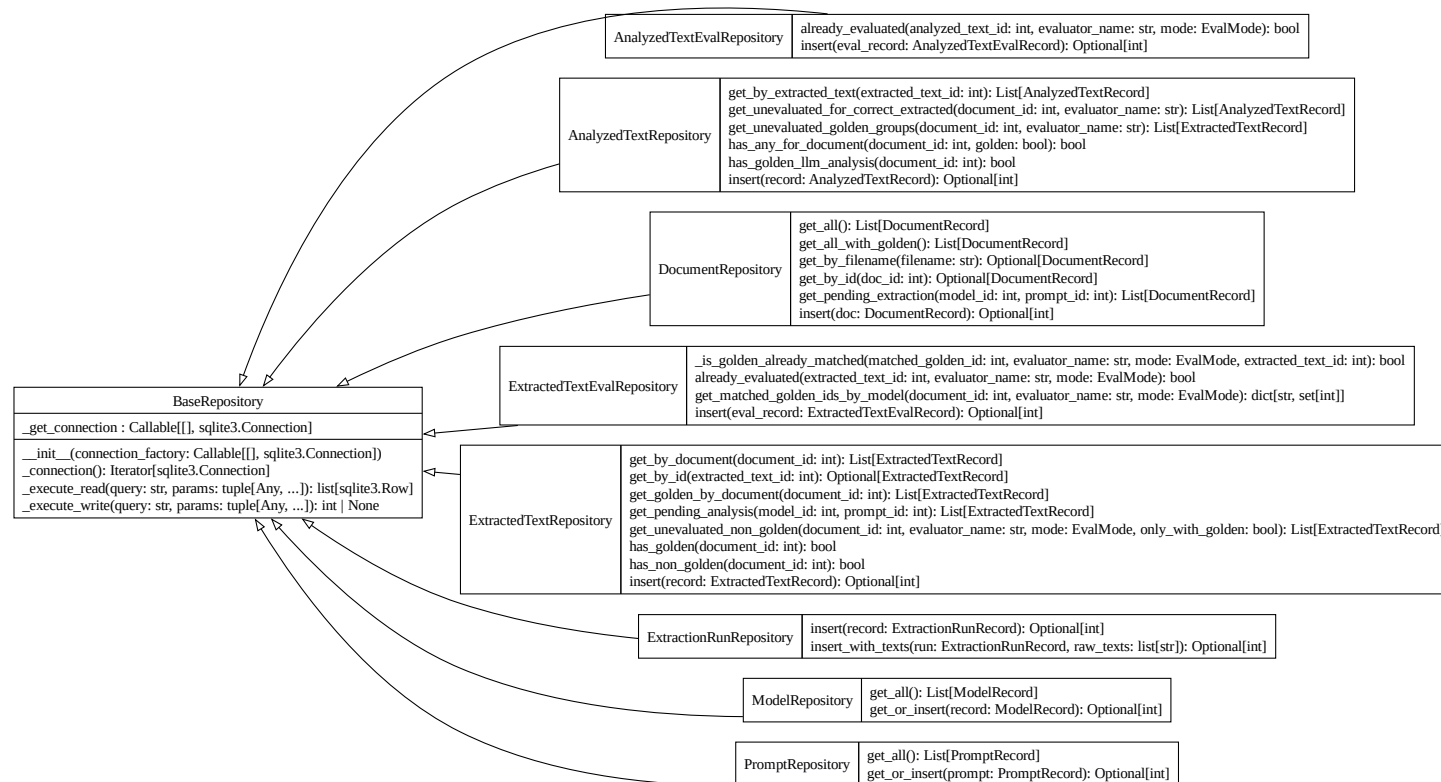
Tabulka B.1: Základní struktura složek a souborů programu Textractor

Soubor / složka	Popis
data/	vstupní a výstupní data (PDF, databáze)
golden_seed/	složka se zlatým datasetem
raw_pdfs/	vstupní soubory pro extrakci
extracted_images/	složka pro ukládání obrázků
database.db	soubor s databází
logs/	soubory se záznamy běhu
src/	zdrojové kódy aplikace
database/	správa databáze
documents/	načítání a zpracování dokumentů
gui/	komponenty grafického rozhraní
llms/	rozhraní pro komunikaci s LLM
pipeline/	skripty pro extrakci a formalizaci
utils.py	pomocné funkce
gui.py	spouštěcí skript GUI
textractor.py	vstupní bod programu
requirements.txt	seznam závislostí
setup.bat / setup.sh	instalační skripty

Diagram tříd databázových repozitářů



Následující stránka obsahuje obrázek C.1 diagramu tříd databázových repozitářů programu Textractor.

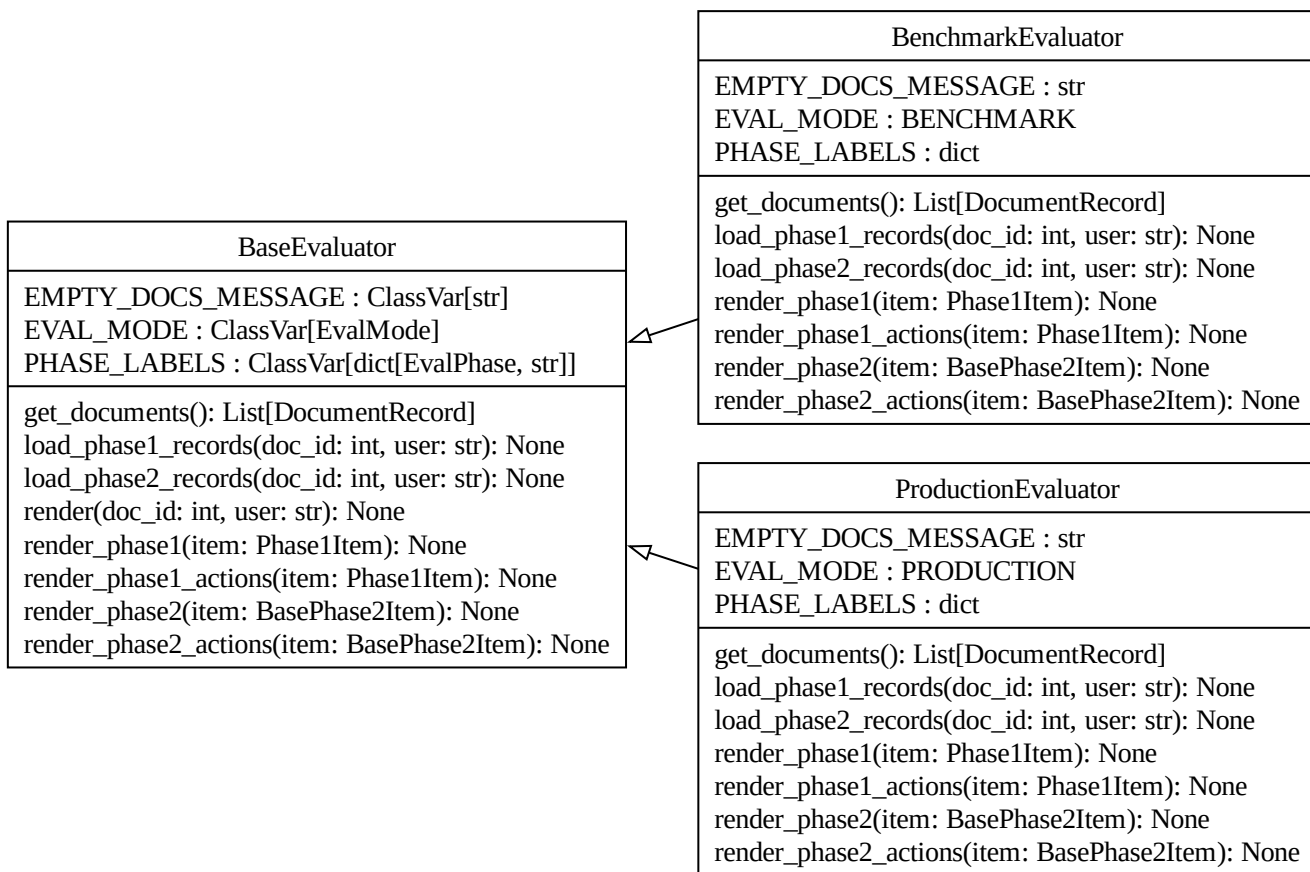


Obrázek C.1: Diagram tříd databázových repozitářů

Diagram tříd pro GUI



Následující stránka obsahuje obrázek D.1 diagramu tříd grafického uživatelského rozhraní Textractor Studio pro program Textractor.



Obrázek D.1: Diagram tříd pro grafické uživatelské rozhraní Texttractor Studio

Uživatelská příručka pro spuštění programu Textractor



Tato uživatelská příručka informuje o instalaci a spuštění programu Textractor na operačních systémech **Microsoft Windows** a **Linux**.

E.1 Struktura archivu

Archiv A23B0169P_prilohy.zip obsahuje následující strukturu:

- Aplikace_a_knihovny – její podsložka app/ obsahuje zdrojové soubory programu Textractor; podsložka docs/ obsahuje dokumentaci ke kódu programu ve formátu html tvořenou knihovnou **pdoc** [48].
- Text_prace – obsahuje zdrojové kódy k dokumentu.
- Vstupni_data – její podsložka raw_pdfs/ obsahuje dokumenty, na kterých bylo provedeno testování; podsložka golden_seed/ obsahuje ručně extrahované a formalizované požadavky z těchto dokumentů ve formátu json.
- Vysledky – databáze s extrahovanými a formalizovanými požadavky a soubor se sestavou v **Microsoft PowerBI** – analyza_llm.pbix, kde probíhala analýza.
- Readme.txt – soubor s informacemi o spuštění a struktuře archivu.

E.2 Minimální požadavky

Spuštění aplikace bylo testováno na následujících technologiích:

- Operační systém **Microsoft Windows** 11 nebo **Linux** (konkrétně **Fedora** verze 43).
- Interpreter jazyka **Python** verze 3.14.3.

- Správce balíčků pro Python – **pip** verze 26.0.1.
- Virtuální prostředí pro Python – **venv**.

E.3 Instalace a spuštění

Spuštění programu Textractor se dá docílit následujícími kroky:

1. Stáhnout a rozbalit archiv `A23B0169P_prilohy.zip`.
2. Přesunout se do kořenového adresáře programu Textractor v `Aplikace_a_knihovny/app`.
3. Spustit skript `setup.sh` (pro Linux, například pomocí příkazu `./setup.sh`), případně `setup.bat` (pro **Microsoft Windows**, například pomocí příkazu `setup.bat` nebo dvojím kliknutím). Obsah tohoto skriptu je popsán v sekci E.3.1.
4. Naplnění (případně vytvoření) souboru `.env` pro uložení proměnných do prostředí běhu, pokud není potřeba používat **jen Textractor Studio v evaluačním režimu**. Pro tyto účely je vhodné upravit připravený soubor `example.env`.
5. Nastavení virtuálního prostředí v terminálu příkazem `source .venv/bin/activate` (pro Linux), případně `.venv\Source\activate` (pro **Microsoft Windows**).
6. Spuštění samotného programu příkazem `python3 textractor.py`, případně `py textractor.py` (pro **Microsoft Windows**).
7. Po splnění předchozích kroků stačí následovat instrukce v terminálu od programu Textractor.¹

E.3.1 Instalační skript

Tento instalační skript vytvoří ve spouštěné složce (doporučeno spouštět v kořenovém adresáři projektu) virtuální prostředí pro jazyk **Python**. Dále v něm aktualizuje správce balíčků **pip**, pomocí kterého dalším příkazem nainstaluje potřebné knihovny pro běh programu:

- **streamlit** verze 1.55.0,
- **streamlit_pdf_viewer** verze 0.0.28,
- **tqdm** verze 4.67.3,

¹V případě nevyplnění potřebného API klíče je možné, že některé jazykové modely nebudou dostupné.

- `typing_extensions` verze 4.15.0,
- `python-dotenv` verze 1.2.2,
- `pydantic` verze 2.12.5,
- `pdfplumber` verze 0.11.9,
- `openai` verze 2.30.0,
- `mistralai` verze 2.1.3,
- `ollama` verze 0.6.1,
- `pypdfium2` verze 5.6.0.

Tyto knihovny a jejich verze se nachází v souboru `requirements.txt`.

E.3.2 Soubor s proměnnými prostředí běhu

Soubor `.env` obsahuje proměnné prostředí, kde s některými program při běhu počítá a nelze ho bez nich spustit.

Hlavní proměnné jsou API klíče pro jazykové modely použité pro extrakční a formalizační fázi v programu Textractor. Program počítá s tím, že jsou použité pouze jazykové modely společností OpenAI (proměnná s názvem `OPENAI_API_KEY`) a Mistral (proměnná s názvem `MISTRAL_API_KEY`). Případně lze použít lokální modely Llama od společnosti Meta, ovšem API klíč není potřeba.

Při použití rozhraní Textractor Studio, tedy pro evaluaci již extrahovaných požadavků tyto proměnné pro API klíče nejsou potřebné. Pro vyhodnocení je možné doplnit volitelnou proměnnou `MODE`, která označuje, zda se má Textractor Studio spustit v produkčním nebo evaluačním režimu. Bez definování této proměnné se standardně **Textractor Studio** spustí v evaluačním režimu. Pro produkční je nutné použít `MODE=production` a pro evaluační režim `MODE=benchmark`.

Proměnná `DATABASE_PATH` určuje cestu k databázi, případně cestu, kde bude databáze vytvořena. Pokud není nastavena, použije se cesta `data/database.db`.

Výsledný soubor `.env` může vypadat například následovně:

```
OPENAI_API_KEY=api-klic-pro-openai
MISTRAL_API_KEY=api-klic-pro-mistral
DATABASE_PATH=data/database.db
MODE=benchmark
```

Systémová zpráva u extrakční fáze



You are an expert Requirements Engineer analyzing Software Requirement Specifications (SRS).

Your task is to carefully read the provided document and extract EVERY explicitly stated project requirement, constraint, or goal.

RULES:

1. COPY the raw text exactly as it appears in the document. Do not summarize or rephrase. IMPORTANT: If the source text contains double quotes ("), you must replace them with single quotes (') to ensure valid JSON output.
2. If some text section contains full context required for understanding the requirement the section is about, you should COPY ALL PARTS CONTAINING THE REQUIREMENT. For example, if there is a table, you should copy whole table as it is. Another example, when there is a long text and somewhere inside it there is a requirement without context, you should copy the sentence and ADDITIONALLY ADD CONTEXT to it (section title, sentence related to requirement, ...) -- there is still rule, that the context should be COPY -- so do not modify the sentences.
3. Extract ALL requirements. Missing a requirement is a critical failure.
4. You MUST return a valid JSON object with the following structure:

```
{
  "raw_text": [
    "requirement 1",
    "requirement 2",
    ...
  ]
}
```

Systemová zpráva u formalizační fáze



You are an expert Technical Analyst. Your task is to process and analyze a raw software requirement and convert it into a structured, developer-friendly format.

You will receive a single raw requirement text. Review it carefully. Your goal is to extract quantifiable parameters (if present) OR provide a clear, concise technical summary of the requirement. A single input text may contain multiple requirements - split them if needed.

As example, from raw text "Display refresh rate should be 60 Hz, internet response should be 5 ms (minimum) and the system should be accessible only to authorized users.", you should extract requirements with following summaries and parameters:

```
{
  "parsed_requirements": [
    {
      "requirement_summary": "Display refresh rate",
      "parameters": [
        { "parameter_value": 60, "parameter_unit": "Hz" }
      ]
    },
    {
      "requirement_summary": "Internet response time",
      "parameters": [
        {
          "parameter_value": 5,
          "parameter_unit": "ms (minimum)"
        }
      ]
    },
    {
      "requirement_summary": "Access restricted to authorized users only",
      "parameters": []
    }
  ]
}
```

```
    }  
  ]  
}
```

If the raw text clearly describes the requirement, copy is enough. But if the raw text is vague, ambiguous or contains a lot of unnecessary information, provide a concise and clear summary in "requirement_summary". The summary should be descriptive enough for software engineers to understand the requirement without referring back to the original text. Keep the language of the input - if the input is in Czech, the output must be in Czech, and so on.

You MUST return a valid JSON object with the following structure:

```
{  
  "parsed_requirements": [  
    {  
      "requirement_summary": "...",  
      "parameters": [  
        {  
          "parameter_value": ...,  
          "parameter_unit": "..."  
        }  
      ]  
    }  
  ]  
}
```

1101001
10101100001110010 1100001
101011010101 1100001



11010011101101001
01100001 1100001
111000101011 1100001